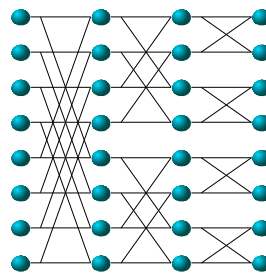


bel_fft

A configurable FFT Co-Processor

Frank Storm

Frank.Storm@gmx.net



Contents

INTRODUCTION.....	6
1 FEATURES.....	7
2 REVISION HISTORY.....	8
3 ARCHITECTURE.....	10
3.1 Data Representation.....	10
3.2 Module bel_butterfly4.....	12
3.3 Module bel_butterfly2.....	15
3.4 Module bel_copy.....	18
3.5 Module bel_fft_core.....	18
4 INTERFACE SIGNALS.....	20
4.1 Memory (Master) Interface.....	20
4.1.1 AXI Version.....	20
4.1.2 Avalon Version.....	22
4.1.3 Wishbone Version.....	22
4.2 Configuration (Slave) Interface.....	23
4.2.1 AXI Version.....	23
4.2.2 Avalon Version.....	24
4.2.3 Wishbone Version.....	25
4.3 Clock and Reset.....	25
4.4 Interrupt.....	26
5 CONFIGURATION REGISTERS.....	27
5.1 Overview.....	27

5.1.1 control 0x00000000.....	28
5.1.2 status 0x00000004.....	28
5.1.3 n 0x00000008.....	29
5.1.4 finadr 0x0000000C.....	29
5.1.5 foutadr 0x00000010.....	29
5.1.6 factors[i] 0x00000014 – 0x00000010 + n * 4.....	29
5.1.7 user 0x00000FF8.....	30
6 CONFIGURATION WIZARD.....	31
6.1 General Tab.....	31
6.2 ConfigurationsTab.....	32
6.3 Target Technology Tab.....	34
6.3.1 Generic.....	34
6.3.2 Altera.....	34
6.3.3 Xilinx.....	35
6.4 Simulation Tab.....	36
6.5 Simulation Data Tab.....	37
6.6 Menu Bar.....	38
6.6.1 Open.....	38
6.6.2 Save.....	38
6.6.3 Quit.....	39
6.6.4 Help → About.....	39
6.7 Configuration File.....	39
7 INTEGRATION IN PLATFORM STUDIO.....	41
8 INTEGRATION IN VIVADO IP-INTEGRATOR.....	48
8.1 Zynq Example Design.....	50
8.2 PCI-Express Example Design.....	56
8.2.1 Building the Hardware Design.....	56
8.2.2 Verifying the PCI-Express Design.....	57
8.2.3 Building the Linux Driver and the Application Program. .	57
9 INTEGRATION IN QSYS-BUILDER.....	60

10 MANUAL ROM GENERATION.....	62
10.1 Altera Quartus.....	62
11 SOFTWARE INTERFACE.....	66
11.1 Direct programming.....	66
11.2 Kiss API.....	69
11.3 Alignment Input and Output Data.....	69
11.4 Settings for Cortex-A9 ACP Port.....	70
12 SIMULATION.....	71
12.1 Simulation with ISim and XSim.....	71
12.2 Simulation with Modelsim.....	71
12.2.1 Setup.....	71
12.2.2 Running the Simulation.....	72
12.2.3 Viewing the Results.....	73
12.2.4 Running Simulation from an Instance inside a Vivado Block Design.....	74
13 PERFORMANCE.....	77
13.1 Altera CycloneIII Implementation.....	77
13.2 Xilinx Zynq Implementation.....	77
14 SETTING PATH VARIABLE IN WINDOWS 7.....	78
15 GNU LESSER GENERAL PUBLIC LICENSE	81
15.1 Version 2.1, February 1999.....	81
15.2 Preamble.....	81
15.3 TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION.....	84
16 LICENSE OF KISS FFT.....	94

Introduction

Hardware implementation of the Kiss FFT

bel_fft is a FFT co-processor that can calculate FFTs with arbitrary radix. It is a hardware implementation of the free software Kiss FFT („Keep it simple, Stupid!“) [KISS]. The target was to allow a simple replacement of the software code with the hardware implementation. Therefore bel_fft comes with a software driver that is compatible with the Kiss FFT routines.

bel_fft also has a modular architecture and allows interfacing different bus architectures. So far AXI [AXI], Altera's Avalon [AVALON] bus as well as the Wishbone bus [WISHBONE] is supported. However, bel_fft's architecture allows an easy adaptation to further bus architectures (e.g. AMBA AHB [AMBA]).

bel_fft is distributed under the GNU Lesser Public License 2.1.

1 | Features

So far the bel_fft co-processor has the following features:

- Mixed-radix FFT co-processor for (so far only radix 4 and radix 2 are supported)
- Master bus interface for memory access
- Slave interface for configuration and control
- Modular architecture
- 32 and 64 bit AXI interface
- 32 bit Avalon and Wishbone interface
- 32 bit and 16 bit data types
- Can be configured to support four different configurations
- Configuration is done via a GUI
- Twiddle factors as ROM(s)
- Interrupt generation
- Software compatible to Kiss FFT
- Seamless integration into Xilinx Platform Studio [XPS]
- Seamless integration into Xilinx Vivado IP-Integrator
- Seamless integration into Altera's QSYS [QSYS]
- Written in Verilog

2 | Revision History

Revision	Date	Changes
0.5	2012/02/12	Initial Version
0.6	2013/01/01	Added AXI interface New target Xilinx Support for ISim and XSim butterfly2 implemented Support for Xilinx XPS Added example design for Xilinx Zynq device
0.7	2013/12/14	Support for Vivado IP-Integrator Added additional AXI interface signals Register for user bus signals Experimental support of Icarus Verilog, Complete example design for Xilinx EDK and Vivado Bugfixes
0.8	2017/05/04	Changed reset from asynchronous to synchronous Fix for FFTs larger than 2K words Update for Vivado 2016.4 Example design with PCI-Express interface for the AC701 board with Linux driver and test program. Replaced previously delivered Vivado project with a script which generates the project.

ARM, ARM Limited, and AMBA are registered trademarks or trademarks of ARM Limited.

Altera, The Programmable Solutions Company, the stylized Altera logo, specific device designations, and all other words and logos that are identified as trademarks and/or service marks are, unless noted otherwise, the trademarks and service marks of Altera Corporation in the U.S. and other countries.

Xilinx, the Xilinx logo, Artix, ISE, Kintex, Spartan, Virtex, Vivado, Zynq are trademarks of Xilinx in the United States and other countries.

All other company or product names herein are the property of their respective owners.

3 | Architecture

The picture below shows the architecture and hierarchy of the main blocks.

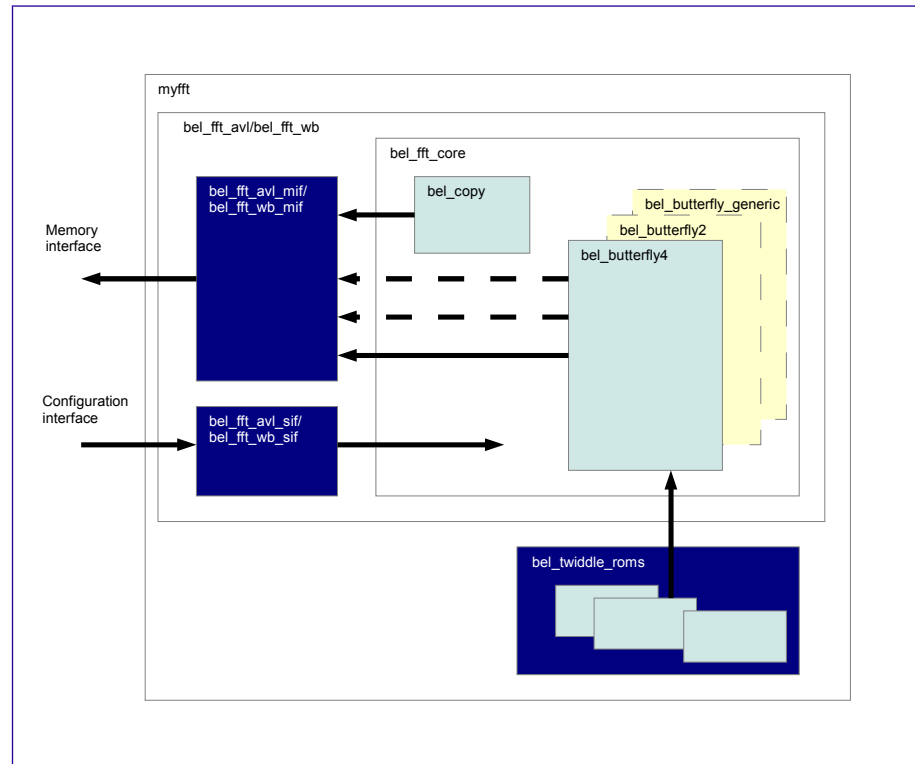


Fig 3.1: *bel_fft* architecture

The top level block *myfft* is generated by the configuration wizard from a template. Depending on the selected interface (AXI, Avalon or Wishbone) the module *bel_fft_axi*, *bel_fft_avl* or the module *bel_fft_wb* is instantiated. In parallel to this module the module *bel_twiddle_roms* is instantiated which is also generated by the configuration wizard. Due to the selected configurations and targets this module instantiates the ROMs from the target technology.

3.1 | Data Representation

The FFT co-processor supports two different data representations in Memory, 16 bit with real and imaginary part in one word and 32 bit

with real and imaginary part in adjacent words. Basically, the internal part of the co-processor is independent of how the data layout in memory looks like. The size of the data is controlled by a generic parameter (word_width). The handling of the data layout is done in the bus interface modules. This makes an adaptation to new bus interfaces (e.g. AMBA AHB or 64 bit wide Avalon buses) easy, because it is only necessary to replace the master bus interface module.

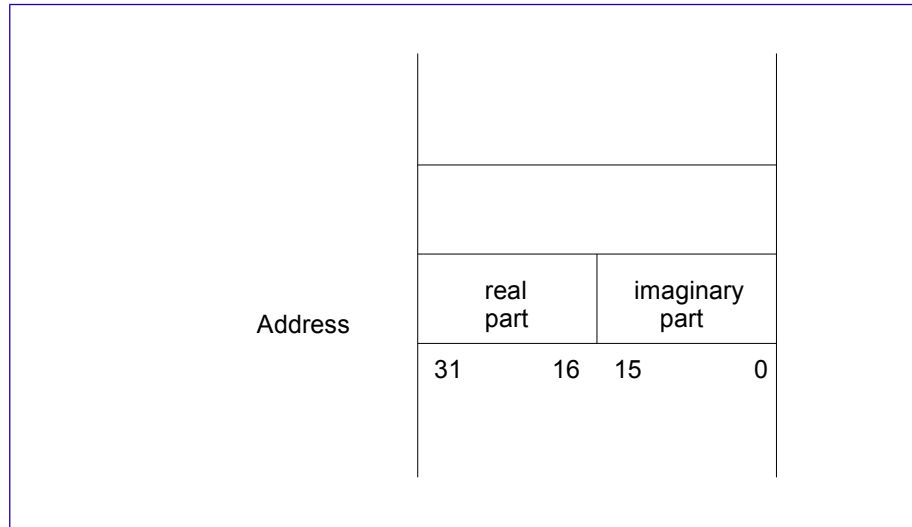


Fig 3.2: 16 bit data representation

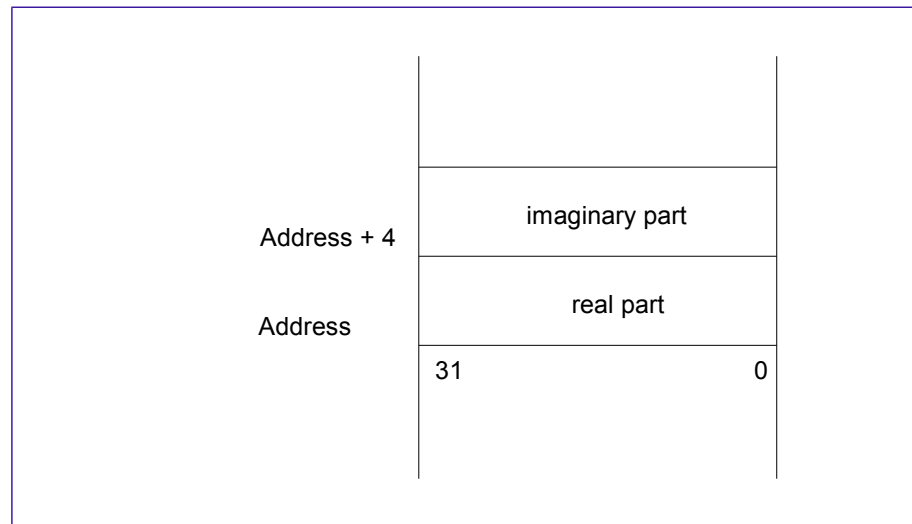


Fig 3.3: 32 bit data representation

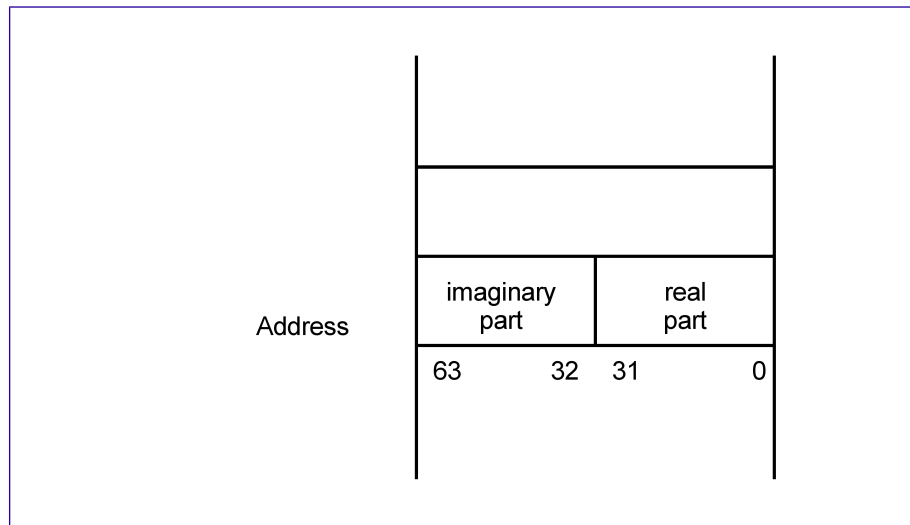


Fig 3.4: 32 bit data representation on a 64 bit bus (AXI-bus, Xilinx Zynq)

3.2 | Module bel_butterfly4

The module `bel_butterfly4` implements the function `kf_bfly4` from the Kiss FFT. It has a memory master interface to load and store the data. The module is started with a start signal and returns with a finish signal.

```
static void kf_bfly4(
    kiss_fft_cpx * Fout,
    const size_t fstride,
    const kiss_fft_cfg st,
    const size_t m
)
{
    kiss_fft_cpx *tw1,*tw2,*tw3;
```

```

kiss_fft_cpx scratch[6];
size_t k=m;
const size_t m2=2*m;
const size_t m3=3*m;

tw3 = tw2 = tw1 = st->twiddles;

do {
    C_FIXDIV(*Fout,4); C_FIXDIV(Fout[m],4);
    C_FIXDIV(Fout[m2],4); C_FIXDIV(Fout[m3],4);

    C_MUL(scratch[0],Fout[m] , *tw1 );
    C_MUL(scratch[1],Fout[m2] , *tw2 );
    C_MUL(scratch[2],Fout[m3] , *tw3 );

    C_SUB( scratch[5] , *Fout, scratch[1] );
    C_ADDTO(*Fout, scratch[1]);
    C_ADD( scratch[3] , scratch[0] , scratch[2] );
    C_SUB( scratch[4] , scratch[0] , scratch[2] );
    C_SUB( Fout[m2], *Fout, scratch[3] );
    tw1 += fstride;
    tw2 += fstride*2;
    tw3 += fstride*3;
    C_ADDTO( *Fout , scratch[3] );

    if(st->inverse) {
        Fout[m].r = scratch[5].r - scratch[4].i;
        Fout[m].i = scratch[5].i + scratch[4].r;
        Fout[m3].r = scratch[5].r + scratch[4].i;
        Fout[m3].i = scratch[5].i - scratch[4].r;
    }else{
        Fout[m].r = scratch[5].r + scratch[4].i;
        Fout[m].i = scratch[5].i - scratch[4].r;
        Fout[m3].r = scratch[5].r - scratch[4].i;
        Fout[m3].i = scratch[5].i + scratch[4].r;
    }
    ++Fout;
}while(--k);
}

```

State	Addr	Div4-Out	CMUL-In A	CMUL-In B	CMUL-Out	CSUB-In	CSUB-Out	CADD-In	CADD-Out	CADDSUB-In	CADDSUB-Out	tw_adr
LOAD2	Fout2-Addr											
LOAD3	Fout3-Addr	Fout2										tw2adr
LOAD1	Fout1-Addr	Fout3	Fout2	tw2								tw3adr
LOAD0	Fout0-Addr	Fout1	Fout3	tw3								tw1adr
EXEC0		Fout0	Fout1	tw1	scratch1							
EXEC1					scratch2	Fout0/scratch1	scratch5	Fout0/scratch1	Fout0			
EXEC2					scratch0							
EXEC3						scratch0/scratch2	scratch4	scratch0/scratch2	scratch3			
EXEC4						Fout0/scratch3	Fout2	Fout0/scratch3	Fout0	scratch4/scratch5	Fout1	
SAVE2	Fout2-Addr									scratch4/scratch5	Fout3	
SAVE0	Fout0-Addr											
SAVE3	Fout3-Addr											
SAVE1	Fout1-Addr											

Fig 3.5: State machine of bel_butterfly4

3.3 | Module bel_butterfly2

The module bel_butterfly2 implements the function kf_bfly2 from the Kiss FFT. The interface is the same as for the module bel_butterfly4.

```
static void kf_bfly2(
    kiss_fft_cpx * Fout,
    const size_t fstride,
    const kiss_fft_cfg st,
    int m
)
{
    kiss_fft_cpx * Fout2;
    kiss_fft_cpx * tw1 = st->twiddles;
    kiss_fft_cpx t;
    Fout2 = Fout + m;
    do{
        C_FIXDIV(*Fout,2); C_FIXDIV(*Fout2,2);

        C_MUL (t, *Fout2 , *tw1);
        tw1 += fstride;
        C_SUB( *Fout2 , *Fout , t );
        C_ADDTO( *Fout , t );
        ++Fout2;
        ++Fout;
    }while (--m);
}
```


State	adr_o	cdlv2-Out	cadd-In A	cadd-In B	cadd-Out	csub-In A	csub-In B	csub-Out	cmul-In A	cmul-In B	cmul-Out	tw_adr
IDLE												
INIT												
LOAD2	foutadr2											
LOAD1	foutadr	fout2										tw1adr
EXEC0		fout							fout2	tw1		
EXEC1												
EXEC2											t	
EXEC3			fout	t	fout	fout	t	fout2				
SAVE2	foutadr2											
SAVE1	foutadr											

Fig 3.6: State machine of bel_bufferfly2

3.4 | Module bel_copy

Module bel_copy implements the data copy loop from the function kf_work of the Kiss FFT. Like module bel_butterfly4 it is also controlled via start and finish signals.

```
do{
    *Fout = *f;
    f += fstride*in_stride;
}while(++Fout != Fout_end );
```

3.5 | Module bel_fft_core

This is the overall control module. It also contains all configuration registers and the modules bel_butterfly4 and bel_copy. This is basically the implementation of the function kf_work of the Kiss FFT.

As you can see from the C code, kf_work is called recursively. This is also implemented in the bel_fft_core modules with stacks for the function parameters. The depth of the recursion is calculated by the configuration wizard and determines the value of the parameter stage_num.

```
static
void kf_work(
    kiss_fft_cpx * Fout,
    const kiss_fft_cpx * f,
    const size_t fstride,
    int in_stride,
    int * factors,
    const kiss_fft_cfg st
)
{
    kiss_fft_cpx * Fout_beg=Fout;
    const int p=*factors++; /* the radix */
    const int m=*factors++; /* stage's fft length/p */
    const kiss_fft_cpx * Fout_end = Fout + p*m;
```

```

if (m==1) {

    do{
        *Fout = *f;
        f += fstride*in_stride;
    }while(++Fout != Fout_end );
else{
    do{
        // recursive call:
        // DFT of size m*p performed by doing
        // p instances of smaller DFTs of size m,
        // each one takes a decimated version of the input
        kf_work( Fout , f, fstride*p, in_stride,
factors,st);
        f += fstride*in_stride;
    }while( (Fout += m) != Fout_end );
    }

    Fout=Fout_beg;

    // recombine the p smaller DFTs
    switch (p) {
        case 2: kf_bfly2(Fout,fstride,st,m); break;
        case 3: kf_bfly3(Fout,fstride,st,m); break;
        case 4: kf_bfly4(Fout,fstride,st,m); break;
        case 5: kf_bfly5(Fout,fstride,st,m); break;
        default: kf_bfly_generic(Fout,fstride,st,m,p); break;
    }
}

```

4 | Interface Signals

The FFT co-processor has four different interfaces

- Memory (master) interface
- Configuration (slave) interface
- Clock and reset interface
- Interrupt sender interface

The memory and the configuration interface are configurable and can have an Avalon or a Wishbone interface.

4.1 | Memory (Master) Interface

4.1.1 | AXI Version

In the current implementation the data width is two times the word size. Real and imaginary part are read and written in one word.

Clock and reset signals are part of the interface. However, since there is no clock synchronization between an interface and an internal clock, the clocks for the master and the slave interface have to be the same clock.

Signal Name	Direction	Description
m_aclk	In	Clock
m_aresetn	in	Reset, low active
m_arready	in	Read address ready
m_arvalid	out	Read address valid
m_araddr[31:0]	out	Read address
m_arprot[2:0]	out	Read protection type

m_arlen[7:0]	out	Read burst length
m_arsize[2:0]	out	Read burst size
m_rready	out	Read data ready
m_rvalid	in	Read data valid
m_rdata[width * 2 - 1:0]	in	Read data
m_rresp[1:0]	in	Read response
m_awready	in	Write address ready
m_awvalid	out	Write address valid
m_awaddr[31:0]	out	Write address
m_awprot[2:0]	out	Write protection type
m_awlen[7:0]	out	Write burst length
m_awsiz[2:0]	out	Write burst size
m_wready	in	Write data ready
m_wvalid	out	Write data valid
m_wdata[width * 2 - 1:0]	out	Write data
m_wstrb[width * 2 / 8 - 1:0]	out	Write data byte strobe
m_bready	out	Write response ready
m_bvalid	in	Write response valid
m_bresp[1:0]	in	Write response
m_arcache[3:0]	out	Read cache
m_aruser[4:0]	out	Read user signals

m_arburst[1:0]	out	Read burst
m_rlast	in	Read data last burst
m_awcache[3:0]	out	Write cache
m_awuser[4:0]	out	Write user signals
m_awburst[1:0]	out	Write burst
m_wlast	out	Write data last burst

4.1.2 | Avalon Version

Signal Name	Direction	Description
m_address[31:0]	out	Address
m_readdata[31:0]	in	Read data
m_writedata[31:0]	out	Write data
m_read	out	Read
m_write	out	Write
m_waitrequest	in	Wait request
m_readdatavalid	in	Read data valid

4.1.3 | Wishbone Version

Signal Name	Direction	Description
mwb_adr_o[31:2]	out	Address
mwb_dat_i[31:0]	in	Read data

mwb_dat_o[31:0]	out	Write data
mwb_sel_o[3:0]	out	Byte select
mwb_cyc_o	out	Cycle in process
mwb_we_o	out	Write enable
mwb_stb_o	out	Data strobe
mwb_ack_i	in	Acknowledge
mwb_err_i	in	Error signal
mwb_stall_i	in	No pipelining

4.2 | Configuration (Slave) Interface

The data width of the slave interface is always 32 bit.

4.2.1 | AXI Version

Signal Name	Direction	Description
s_aclk	in	Clock
s_aresetn	in	Reset, low active
s_awaddr[7:0]	in	Write address
s_awvalid	in	Write address valid
s_wdata[31:0]	in	Write data
s_wstrb[3:0]	in	Write data byte strobe
s_wvalid	in	Wait data valid

s_bready	in	Write response ready
s_araddr[7:0]	in	Read address
s_arvalid	in	Read address valid
s_rready	in	Read data ready
s_arready	out	Read address ready
s_rdata[31:0]	out	Read data
s_rresp[1:0]	out	Read data response
s_rvalid	out	Read data valid
s_wready	out	Write data ready
s_bresp[1:0]	out	Write response
s_bvalid	out	Write response valid
s_awready	out	Write address ready

4.2.2 | Avalon Version

Signal Name	Direction	Description
s_address[5:0]	in	Address
s_readdata[31:0]	out	Data output
s_writedata[31:0]	out	Data input
s_read	in	Read enable
s_write	in	Write enable
s_byteenable[3:0]	in	Byte select
s_waitrequest	out	Wait request
s_readdatavalid	out	Read data valid

4.2.3 | Wishbone Version

Signal Name	Direction	Description
swb_adr_i[5:0]	in	Address
swb_dat_i[31:0]	in	Data input
swb_dat_o[31:0]	out	Data output
swb_sel_i[3:0]	in	Byte select
swb_we_i	in	Write enable
swb_stb_i	in	Data strobe
swb_cyc_i	in	Cycle in process
swb_ack_o	out	Acknowledge
swb_err_o	out	Error

4.3 | Clock and Reset

Signal Name	Direction	Description
clk_i	in	Clock
rst_i	in	Reset (high active)

4.4 | Interrupt

Signal Name	Direction	Description
int_o	out	Interrupt (high active)

5 | Configuration Registers

5.1 | Overview

Address	Width	Name	Description
0x00000000	31:0	control	Control register
0x00000004	31:0	status	Status register
0x00000008	31:0	n	FFT size register
0x0000000C	31:0	finadr	Source address register
0x00000010	31:0	foutadr	Destination address register
0x00000014	31:0	factors[0]	p and m factor registers
0x00000018	31:0	factors[1]	
0x00000010 + n * 4	31:0	factors[n - 1]	
0x0000FF8	31:0	user	For the control of special bus signals, like user or cache signals

The number of factors registers depends on the factorization and is calculated by the configuration wizard.

Bits which are not used are not guaranteed to have a defined value.

5.1.1 | control 0x00000000

Bits	Name	Type	Reset Value	Function
0	start	write	0	Starts the FFT
7:1				Not used
8	inten	read, write	0	Enables the interrupt
15:9				Not used
16	inv	read, write	0	Normal or inverse FFT

5.1.2 | status 0x00000004

Bits	Name	Type	Reset Value	Function
0	running	read	0	Indicates whether the FFT is running
1	ov	read	0	Overflow flag
2	finish	read	0	The FFT has finished. Reading a finish bit with value '1' resets the bit
3	err	read	0	Error flag. This flag indicates that an error occurred while the FFT was accessing the bus. This flag is cleared after reading.

Bits	Name	Type	Reset Value	Function
31:4				Not used

5.1.3 | n 0x00000008

Bits	Name	Type	Reset Value	Function
15:0	n	read, write	0	Size of the FFT
31:0				Not used

5.1.4 | finadr 0x0000000C

Bits	Name	Type	Reset Value	Function
31:0	finadr	read, write	0	Source address of the data

5.1.5 | foutadr 0x00000010

Bits	Name	Type	Reset Value	Function
31:0	foutadr	read, write	0	Destination address of the data

5.1.6 | factors[i] 0x00000014 – 0x00000010 + n * 4

Bits	Name	Type	Reset Value	Function
15:0	m	read, write	0	Stages FFT length/p

Bits	Name	Type	Reset Value	Function
31:16	p	read, write	0	The radix

5.1.7 | user 0x00000FF8

So far, this register is only used when the memory interface is an AXI interface. These signals have to be set correctly when interfacing the FFT coprocessor with the ACP of a Cortex A9.

Bits	Name	Type	Reset Value	Function
4:0	ruser	read, write	0	Value of the m_ruser signal
7:5	rprot	read, write	0	Value of the m_arprot signal
11:8	rcache	read, write	0	Value of the m_rcache signal
15:12				Not used
20:16	wuser	read, write	0	Value of the m_wuser signal
23:21	wprot	read, write	0	Value of the m_awprot signal
27:24	wcache	read, write	0	Value of the m_wcache signal
31:28				Not used

6 | Configuration Wizard

The FFT co-processor comes with a configuration wizard with a graphical user interface. The wizard is written in Java and was developed with the NetBeans IDE [NETBEANS].

The wizard contains five tabs to do all the settings. After all settings are made the IP is generated by clicking the *Generate* button.

6.1 | General Tab

The wizard creates a project directory for each generated co-processor. This project directory will contain all files that belong to the co-processor and that are generated during ROM generation.

For each generated co-processor a wrapper is generated that instantiates the actual co-processor and does all the parameter settings. The wrapper itself does not have any parameters. This simplifies the integration into an own project, but adds an additional hierarchy layer. If you don't want this additional layer you can simply cut out the instantiation of the co-processor and use that instead.

In addition to the module name and the project directory the bus interface and the width of the data (16 or 32 bit) is specified.

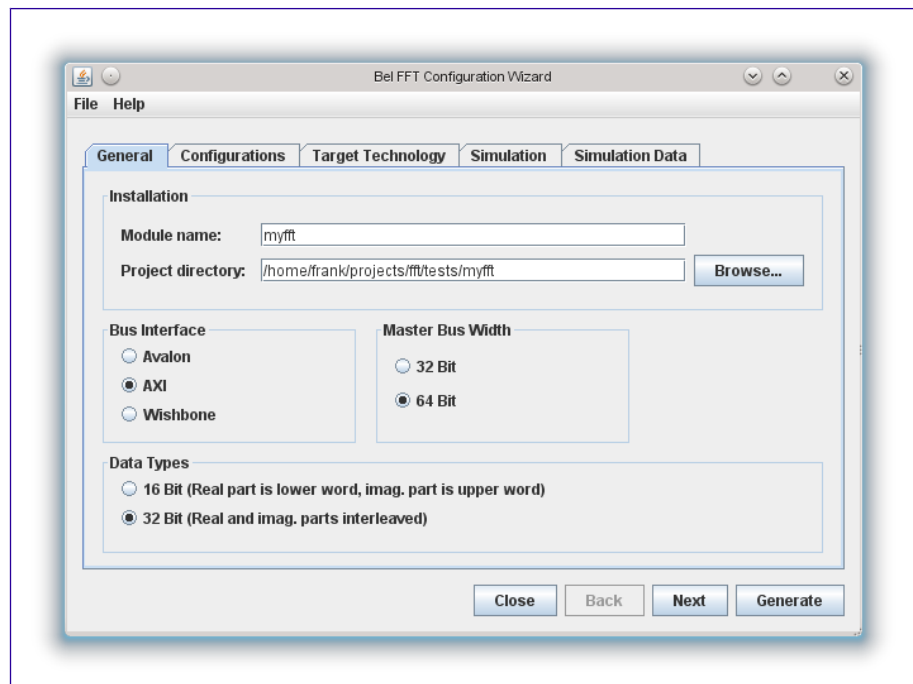


Fig 6.1: General tab

6.2 | ConfigurationsTab

Up to four different configuration are possible with the configuration wizard. For each configuration a different size and whether the FFT is a normal FFT or an inverse FFT. For each configuration a separate ROM is used.

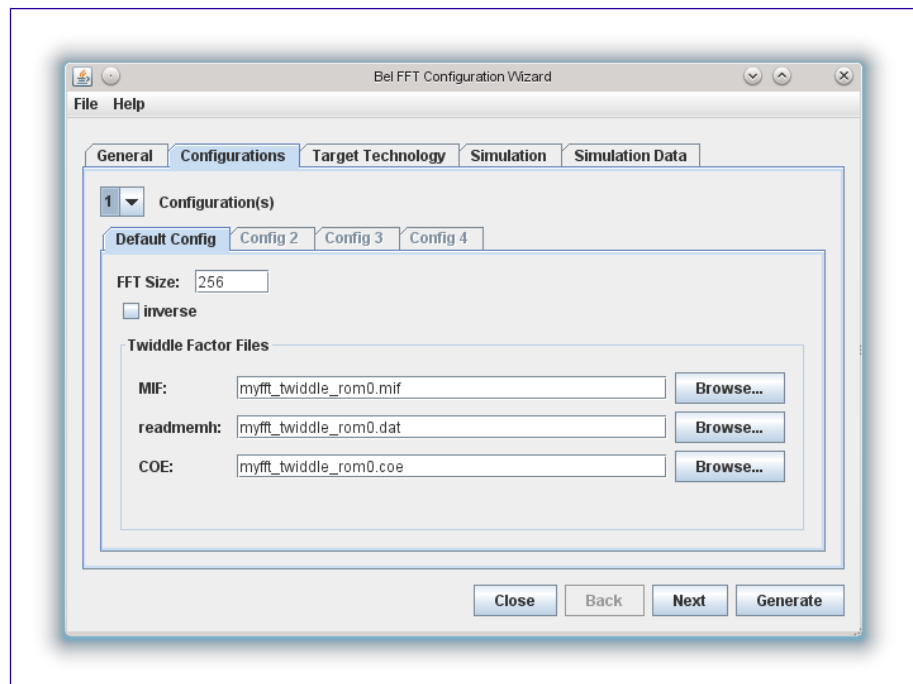


Fig 6.2: Configurations tab

The contents of the twiddle ROMs is generated in three different formats

- MIF [MIF] for the Megawizard of QuartusII
- readmemh format for a standard Verilog simulator
- COE for Xilinx Coregen

The name of the contents file is derived from the module name. Normally it is not necessary to edit the name. However if it is desired to store the files in a different directory under a different name, then it is possible to specify the file names explicitly. If the project name is changed these file names will be updated, but only if no absolute file name is entered.

More than four configurations are possible but are not supported by the configuration wizard. This requires a manual modification of the Verilog source code.

6.3 | Target Technology Tab

Under this tab the target technology can be selected. Currently only two technologies are supported: Generic and Altera FPGAs. So far this also only effects the generation of the twiddle factor ROMs.

6.3.1 | Generic

If generic is used, a technology independent representation is used that may or may not be synthesized by the used synthesis tools and the used technology library.

6.3.2 | Altera

Due to bugs in earlier versions of the qmegawiz program this only works reliable for version 11.0 and 12.X.

The configuration wizard can call the Megawizard from QuartusII in batch mode to generate the twiddle factor ROMs for the required configurations. This is done by calling the QuartusII program qmegawiz for each ROM with the necessary parameters. The target FPGA family has to be selected.

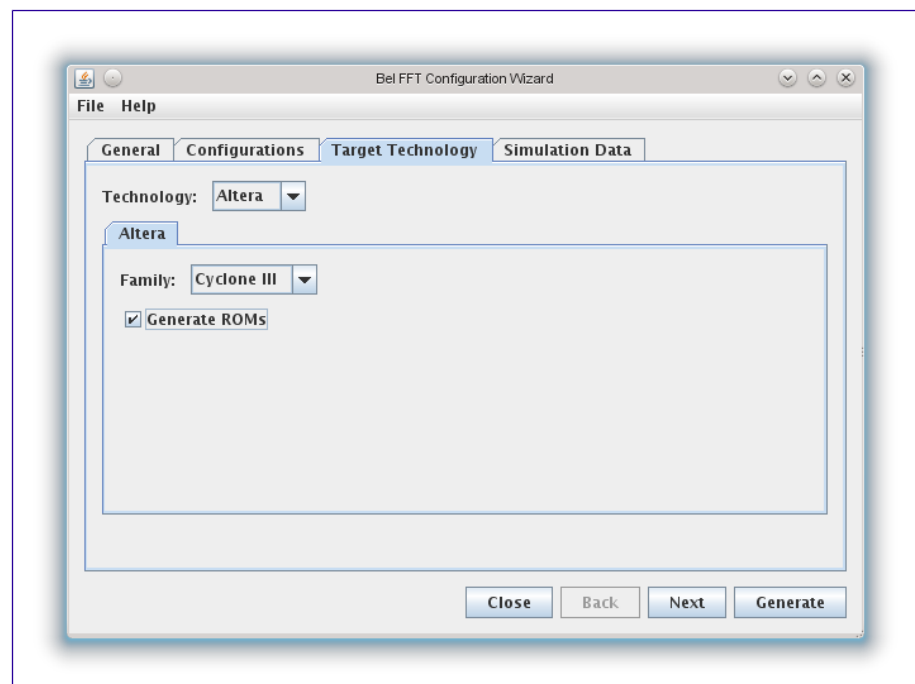


Fig 6.3: Target Technology Tab

If qmegawiz cannot be found because it is not in the search path you will get the following error message.

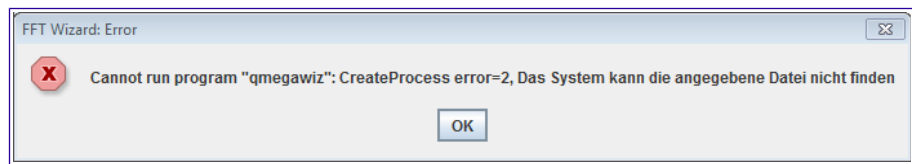


Fig 6.4: qmegawiz not found

In this case you have to expand the search path in your Windows/Linux environment.

6.3.3 | Xilinx

When generating the Coregen project file, the target HDL has to be Verilog!

When Xilinx is selected as target technology the configuration wizard creates all necessary files for a seamless integration into EDK. The flow is described in chapter 7.

To generate the twiddle ROMs for a Xilinx device it is necessary to generate a Coregen Project File for the target device first. This is then used to start Coregen in batch and generate the twiddle ROMs.

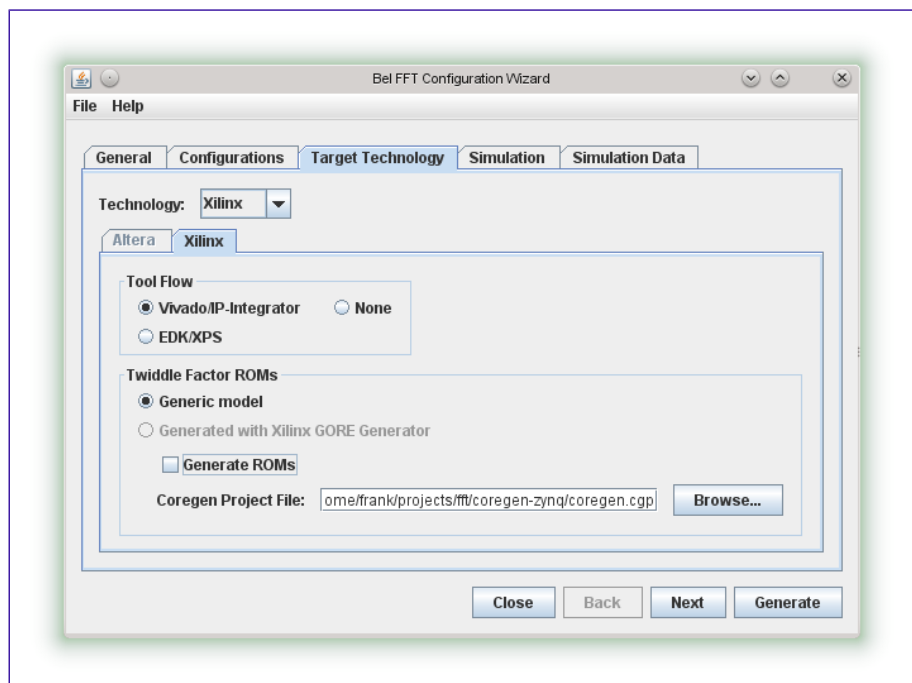


Fig 6.5: Target technology tab - Xilinx

The Coregen project file is copied into the IP subdirectory which is used as the run-directory for Coregen.

There is a problem when using Coregen from Xilinx WebPACK 14.4. The WebPACK installation prevents the correct loading of the required device families. This is a known bug, described under [AR53695] and can be fixed by doing either a full install of the ISE software or by setting the environment variable `XIL_CG_LOAD_ALL_FAMILIES=true`.

If coregen cannot be found because it is not in the search path you will get the following error message.

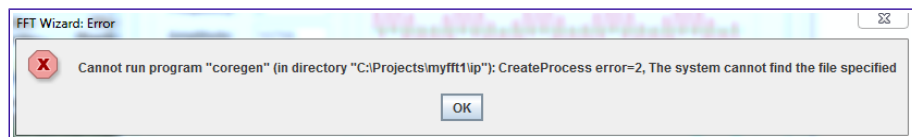


Fig 6.6: coregen not found.

In this case you have to expand the search path in your Windows/Linux environment (See chapter *Setting Path Variable in Windows 7*).

6.4 | Simulation Tab

Currently the configuration wizard creates simulation scripts or setup files for three different simulators:

- Xilinx XSim
- Xilinx ISim
- Mentor Modelsim
- Icarus Verilog

Xilinx ISim and Xsim are only supported when Xilinx or Generic is selected as target technology.

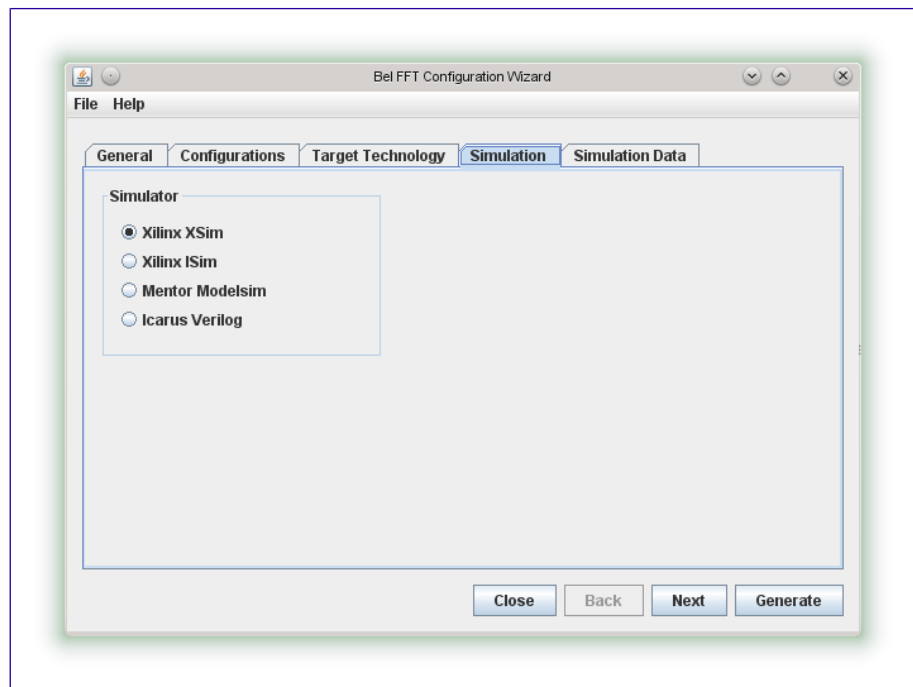


Fig 6.7: Simulation Tab

6.5 | Simulation Data Tab

The configuration wizard also generates a testbench for each configuration. In addition it generates some input data that can be processed by the FFT. The input data consists of an overlay of two different sine waves. The frequency and the amplitude of both sine waves can be entered. The two diagrams show the actual input data and the resulting output data.

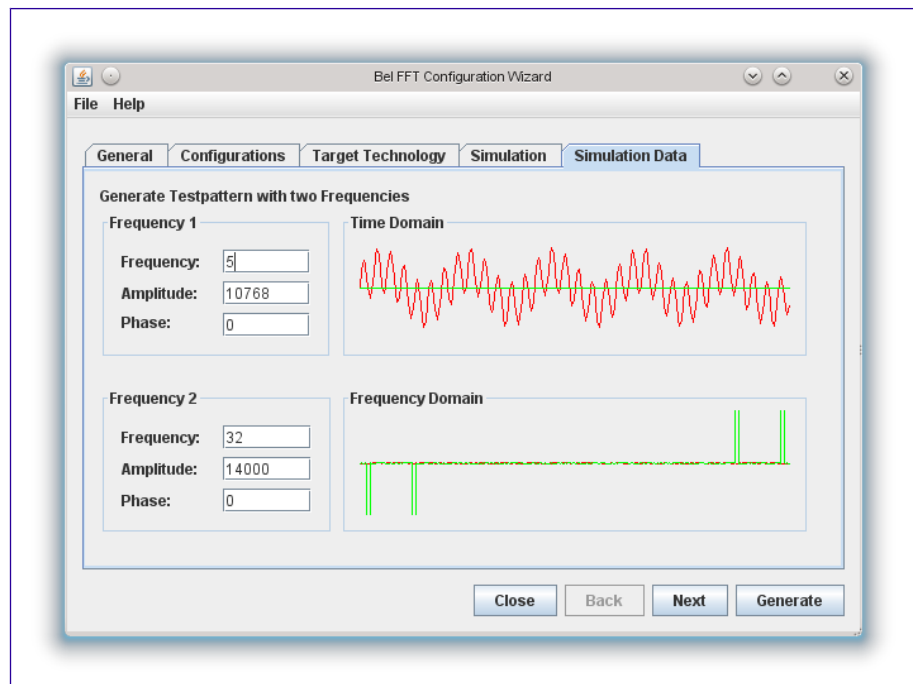


Fig 6.8: Simulation Data tab

6.6 | Menu Bar

6.6.1 | Open...

Reads in previously stored settings. Selecting this menu item opens a file selector to select a XML file with the settings.

6.6.2 | Save...

All settings can be saved as an XML file. Selecting the Save... menu item opens a file selector which lets you specify the name of the configuration file.

6.6.3 | Quit

Leaves the program. The current settings are stored in the current directory in the file `.belwizardrc.xml`.

6.6.4 | Help → About

Under this menu item information about author, version, and license can be found.



Fig 6.9: About dialogue

6.7 | Configuration File

All settings are stored as a XML file. Here is an example of such an XML file.

```
<?xml version="1.0" encoding="UTF-8"?>
<belfft:bel
xmlns:belfft="https://sourceforge.net/projects/belfft">
  <belfft:modulename>myfft1</belfft:modulename>
  <belfft:projectdir>myfft</belfft:projectdir>
  <belfft:datawidth>32</belfft:datawidth>
  <belfft:interface>avalon</belfft:interface>
  <belfft:configurations>
```

```

<belfft:configuration>
  <belfft:size>256</belfft:size>
  <belfft:inverse>false</belfft:inverse>
  <belfft:twiddlefactorfile format="readmemh">twiddles.-
dat</belfft:twiddlefactorfile>
  <belfft:twiddlefactorfile format="mif">twid-
dles.mif</belfft:twiddlefactorfile>
</belfft:configuration>
</belfft:configurations>
<belfft:target>
  <belfft:generateroms>false</belfft:generateroms>
</belfft:target>
<belfft:simulation>
  <belfft:inputsignal>
    <belfft:frequency>5</belfft:frequency>
    <belfft:amplitude>10768</belfft:amplitude>
  </belfft:inputsignal>
  <belfft:inputsignal>
    <belfft:frequency>32</belfft:frequency>
    <belfft:amplitude>14000</belfft:amplitude>
  </belfft:inputsignal>
</belfft:simulation>
</belfft:bel>

```

When the wizard is started it looks for the file `.belwizardrc.xml` in the current directory. If such a file exists the settings are loaded.

7 | Integration in Platform Studio

If the FFT coprocessor is created for the integration into Xilinx Platform Studio (XPS) it is recommended to put it under an IP repository. The name of the project directory should have the form “<module-name>_v<version>_<sub-version>_<patch-level>” (e.g. “myfft_v1_00_a”).

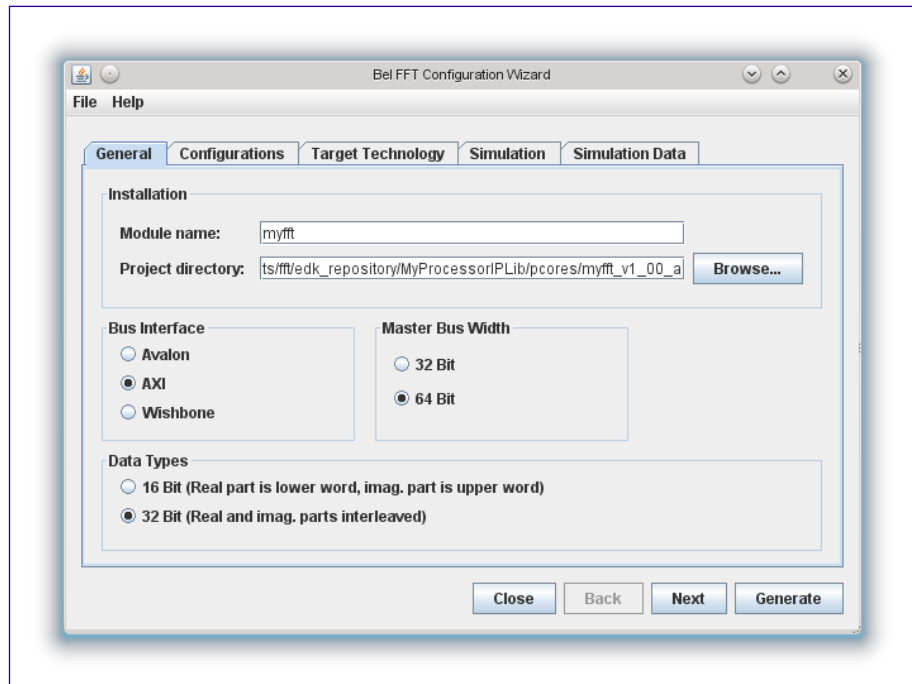


Fig 7.1: Project directory for integration in XPS

The following shows the generated directory structure (myfft_v1_00_a was generated under an existing directory MyProcessorIPLib/pcores)

```
edk_repository/
├── MyProcessorIPLib/
│   ├── drivers/
│   └── pcores/
│       └── myfft_v1_00_a/
│           ├── data/
│           │   ├── myfft_v2_1_0.bbd
│           │   ├── myfft_v2_1_0.mpd
│           │   └── myfft_v2_1_0.pao
│           └── dev1/
│               └── simulation_256/
│                   ├── bel_axi_ram.v
│                   └── input_data_256.dat
```

```

├── isim-simulate.tcl
├── testbench_256.v
├── hdl/
│   └── verilog/
│       ├── bel_butterfly2.v
│       ├── bel_butterfly4.v
│       ├── bel_caddsub.v
│       ├── bel_cadd.v
│       ├── bel_cdiv2.v
│       ├── bel_cdiv4.v
│       ├── bel_cmac.v
│       ├── bel_cmul.v
│       ├── bel_copy.v
│       ├── bel_csub.v
│       ├── bel_fft_axi_mif.v
│       ├── bel_fft_axi_sif.v
│       ├── bel_fft_axi.v
│       ├── bel_fft_core.v
│       ├── bel_fft_def.v
│       ├── myfft_twiddle_rom0.v
│       ├── myfft_twiddle_roms.v
│       └── myfft.v
├── ip/
│   ├── coregen.cgc
│   ├── coregen.cgp
│   ├── coregen.log
│   └── myfft_twiddle_rom0/
│       ├── blk_mem_gen_v7_2_readme.txt
│       ├── doc/
│       │   ├── blk_mem_gen_v7_2_vinfo.html
│       │   └── pg058-blk-mem-gen.pdf
│       ├── example_design/
│       │   ├── myfft_twiddle_rom0_exdes.ucf*
│       │   ├── myfft_twiddle_rom0_exdes.vhd*
│       │   ├── myfft_twiddle_rom0_exdes.xdc*
│       │   └── myfft_twiddle_rom0_prod.vhd*
│       ├── implement/
│       │   ├── implement.bat*
│       │   ├── implement.sh*
│       │   ├── planAhead_ise.bat*
│       │   ├── planAhead_ise.sh*
│       │   ├── planAhead_ise.tcl*
│       │   ├── xst.prj*
│       │   └── xst.scr*
│       ├── simulation/
│       │   ├── addr_gen.vhd*
│       │   ├── bmg_stim_gen.vhd*
│       │   ├── bmg_tb_pkg.vhd*
│       │   ├── functional/
│       │   │   ├── simcmds.tcl*
│       │   │   ├── simulate_isim.sh*
│       │   │   ├── simulate_mti.bat*
│       │   │   ├── simulate_mti.do*
│       │   │   ├── simulate_mti.sh*
│       │   │   ├── simulate_ncsim.sh*
│       │   │   ├── simulate_vcs.sh*
│       │   │   ├── ucli_commands.key*
│       │   │   ├── vcs_session.tcl*
│       │   │   ├── wave_mti.do*
│       │   │   └── wave_ncsim.sv*
│       │   ├── myfft_twiddle_rom0_synth.vhd*
│       │   └── myfft_twiddle_rom0_tb.vhd*

```

```

├── random.vhd*
├── timing/
│   ├── simcmds.tcl*
│   ├── simulate_isim.sh*
│   ├── simulate_mti.bat*
│   ├── simulate_mti.do*
│   ├── simulate_mti.sh*
│   ├── simulate_ncsim.sh*
│   ├── simulate_vcs.sh*
│   ├── ucli_commands.key*
│   ├── vcs_session.tcl*
│   ├── wave_mti.do*
│   └── wave_ncsim.sv*
├── myfft_twiddle_rom0.asy
├── myfft_twiddle_rom0.coe
├── myfft_twiddle_rom0.dat
├── myfft_twiddle_rom0_flist.txt
├── myfft_twiddle_rom0.gise
├── myfft_twiddle_rom0.mem
├── myfft_twiddle_rom0.mif
├── myfft_twiddle_rom0.ngc
├── myfft_twiddle_rom0.v
├── myfft_twiddle_rom0.veo
├── myfft_twiddle_rom0.xco
├── myfft_twiddle_rom0.xise
├── myfft_twiddle_rom0_xmdf.tcl
├── summary.log
├── tmp/
│   ├── _cg/
│   ├── myfft_twiddle_rom0.lso
│   └── _xmsgs/
│       ├── pn_parser.xmsgs
│       └── xst.xmsgs
├── xlnx_auto_0_xdb/
├── netlist/
└── myfft_twiddle_rom0.ngc

```

In XPS the local IP repository has to be added.

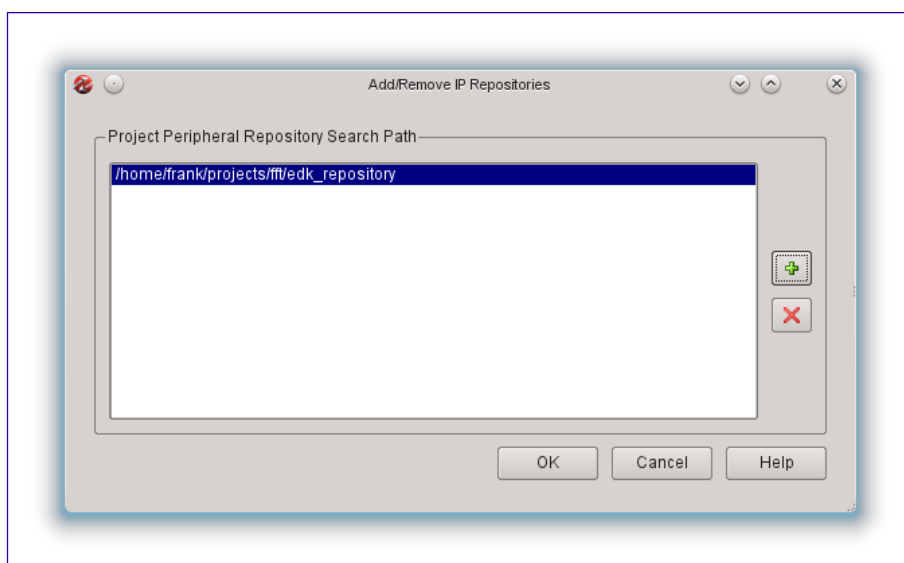


Fig 7.2: Adding the local IP repository

After rescanning the IPs our FFT co processor is listed in the IP catalog and can be used.

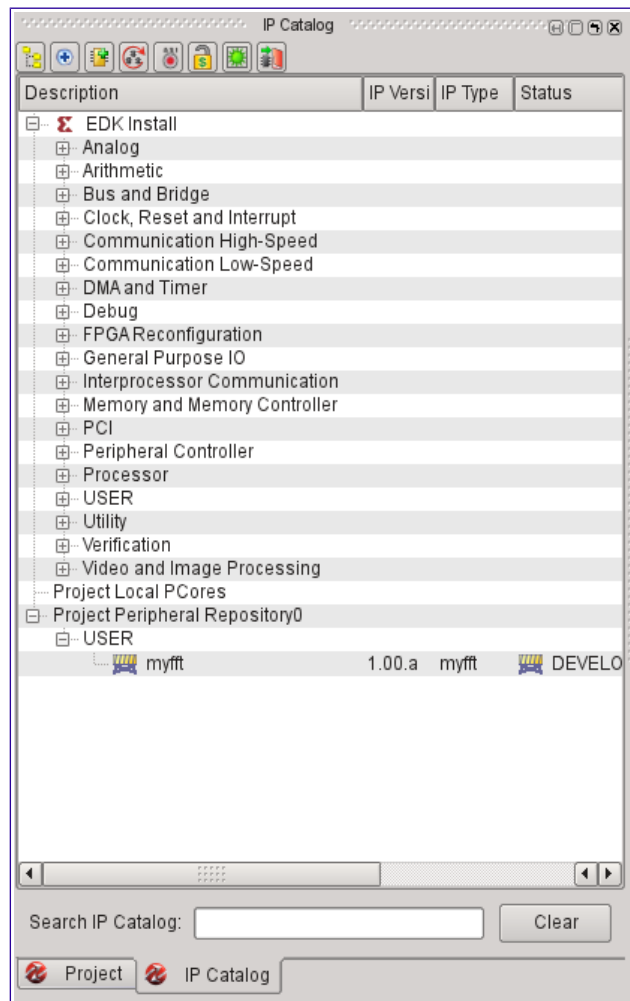


Fig 7.3: IP catalog with myfft

When myfft is instantiated the configuration wizard opens. Since we have everything pre-configured with the bel_fft configuration wizard there is not much left for configuration except base and high address and instance name.

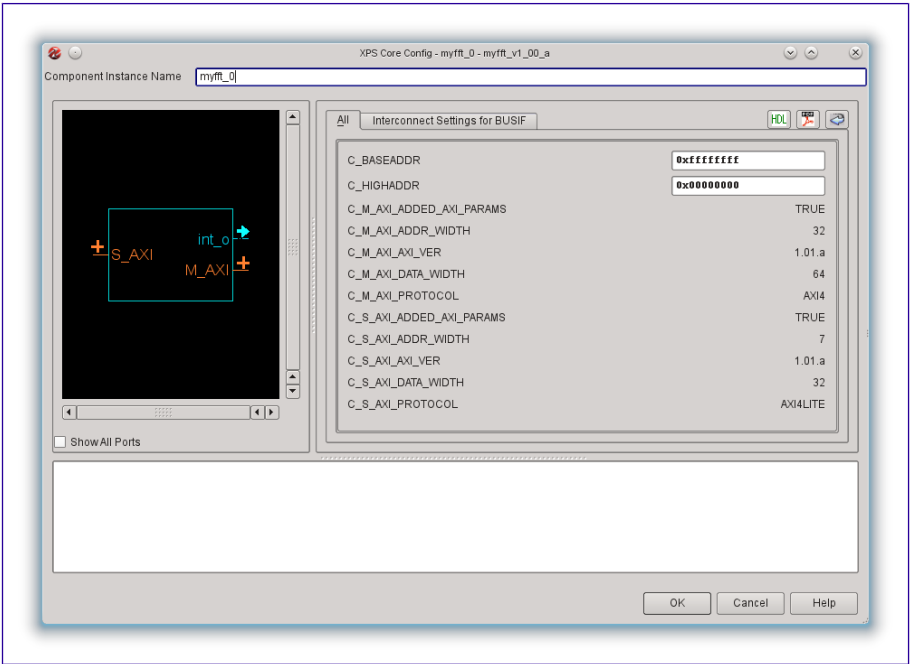


Fig 7.4: Instantiating myfft

The picture below shows a Zynq system (standard configuration) with the slave port of myfft connected to GP0 and the master port connected to the high-speed port 0. The example design can be found in the example directory.

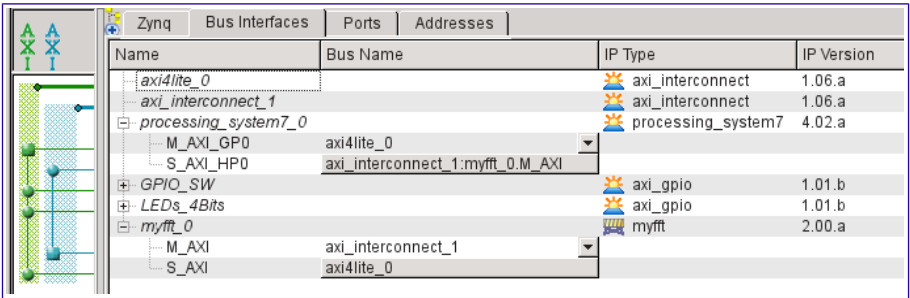


Fig 7.5: Zynq system with bel_fft co-processor

Keep in mind that the clocks of the master and slave interfaces have to be the same!

Apart from connecting the master and slave interfaces the clocks have to be connected to the system. In our example we use clock 0 from the processing system (PS).

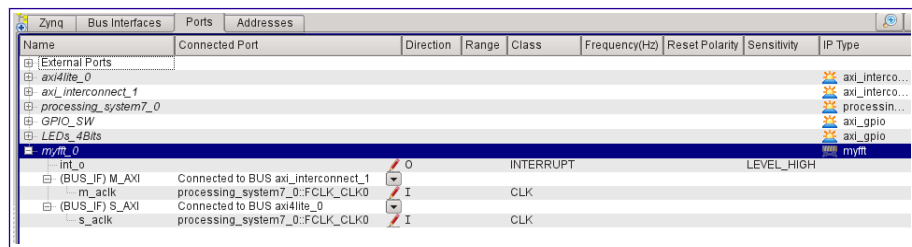


Fig 7.6: Connecting the clocks of myfft

The addresses for myfft are assigned automatically. By default a 64KB address space is used.

Instance	Base Name	Base Address	High Address	Size	Bus Interface(s)	Bus Name	Lock
processing_system7_0's Address Map							
processing_system7_0	C_DDR_RAM_BASEADDR	0x00000000	0x3FFFFFFF	1G			
LEDs_4Bits	C_BASEADDR	0x41200000	0x4120FFFF	64K	S_AXI	axi4lite_0	
GPIO_SW	C_BASEADDR	0x41220000	0x4122FFFF	64K	S_AXI	axi4lite_0	
myfft_0	C_BASEADDR	0x7E400000	0x7E40FFFF	64K	S_AXI	axi4lite_0	
processing_system7_0	C_UART1_BASEADDR	0xE0001000	0xE0001FFF	4K			
processing_system7_0	C_I2C0_BASEADDR	0xE0004000	0xE0004FFF	4K			
processing_system7_0	C_CAN0_BASEADDR	0xE0008000	0xE0008FFF	4K			
processing_system7_0	C_GPIO_BASEADDR	0xE000A000	0xE000AFFF	4K			
processing_system7_0	C_ENET0_BASEADDR	0xE000B000	0xE000BFFF	4K			
processing_system7_0	C_SDIO0_BASEADDR	0xE0100000	0xE0100FFF	4K			
processing_system7_0	C_USB0_BASEADDR	0xE0102000	0xE0102FFF	4K			
processing_system7_0	C_TTC0_BASEADDR	0xE0104000	0xE0104FFF	4K			

Fig 7.7: Address space with myfft

The example design can be found under
example/xilinx/edk/myfft_test_edk.zip.

8 | Integration in Vivado IP-Integrator

With Vivado Xilinx has moved to the IP-XACT format for describing IPs. This is an XML file which contains all information needed: interfaces, files registers, etc. The configuration wizard also creates such an IP-XACT file.

In Vivado it is possible to specify multiple IP repositories. So when configuring an FFT co-processor for the use in Vivado, the project directory has to be placed in such an IP repository.

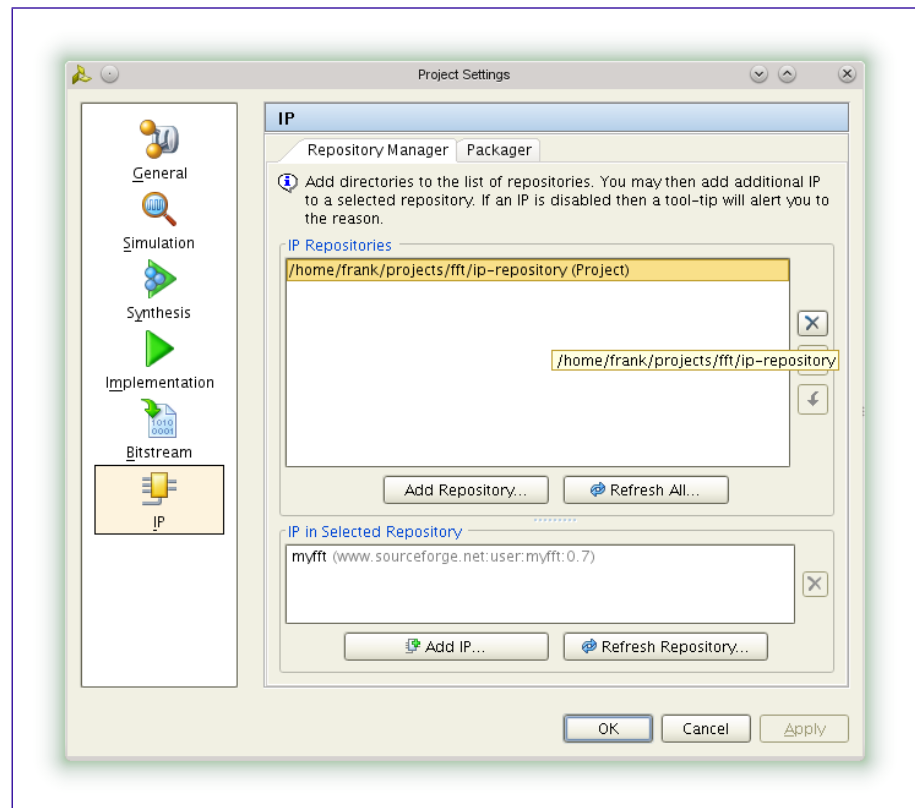


Fig 8.1: Adding the IP repository to a project

So after the configuration of the co-processor the directory containing the project directory has to be added as an IP repository.

After this has been done the co-processor is ready for being used in IP-Integrator.

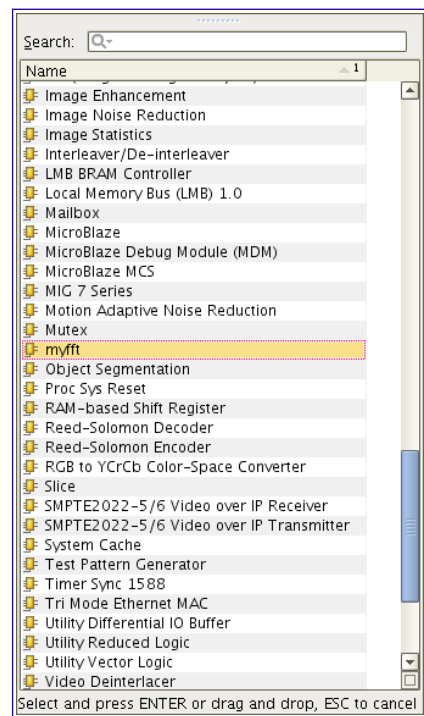


Fig 8.2: IP list with co-processor

Keep in mind that the clocks of the master and slave interfaces have to be the same!

The picture below shows a simple system which connects the co-processor to the processing system of the Zynq.

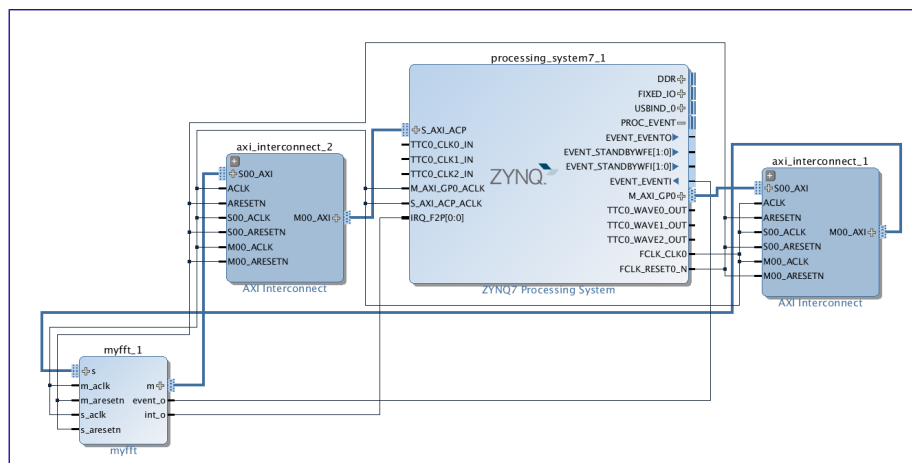


Fig 8.3: Co-processor connected to the PS

8.1 | Zynq Example Design

This example design shows how to connect the FFT core to the ACP port of the Zynq processor system. Starting with version 0.8 this example is no longer distributed as a complete Vivado project but as a script to rebuild the design.

The following is a recipe to rebuild the project and run the software:

1. Copy the whole directory “myfft_zed” to a place of your choice.
2. Start the FFT wizard, open the configuration file “fft256_32.xml” (*File*→*Open*), adjust the project directory so that the “myfft” directory is inside the “myfft_zed” directory and then click *Generate*.
3. Now start Vivado and change to the myfft_zed directory on the Td console (e.g. `cd C:/Projects/myfft_zed`).
4. Start the Td Script (`source create_design.tcl`). Vivado now creates the project, the block design, and performs synthesis, implementation down to the generation of the bit-stream. Figure 8.4 shows the generated hardware design.

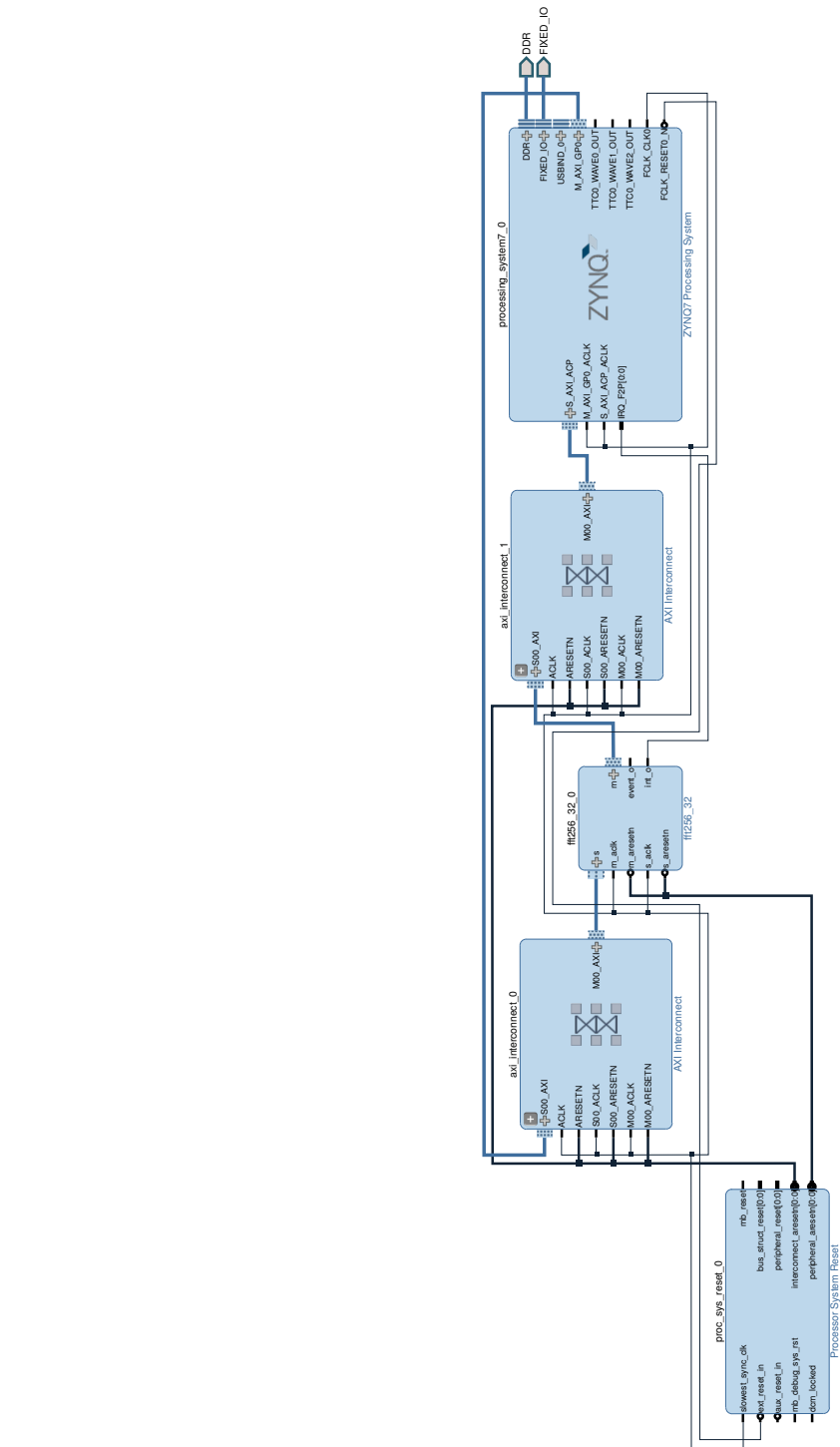


Fig 8.4: Block design of FFT connected to Zynq processor system

5. Export the hardware including bitstream. Then launch the SDK from Vivado.
6. Create a new application project with BSP. Name the project “myfft_test” and leave the default settings unchanged. For the template use “Empty Application”.

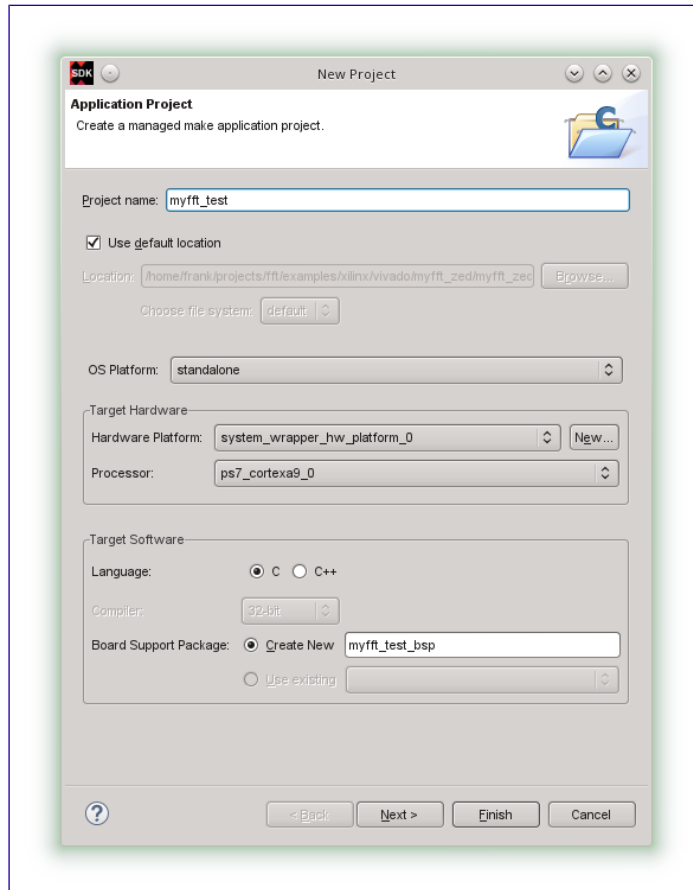


Fig 8.5: New Application Project

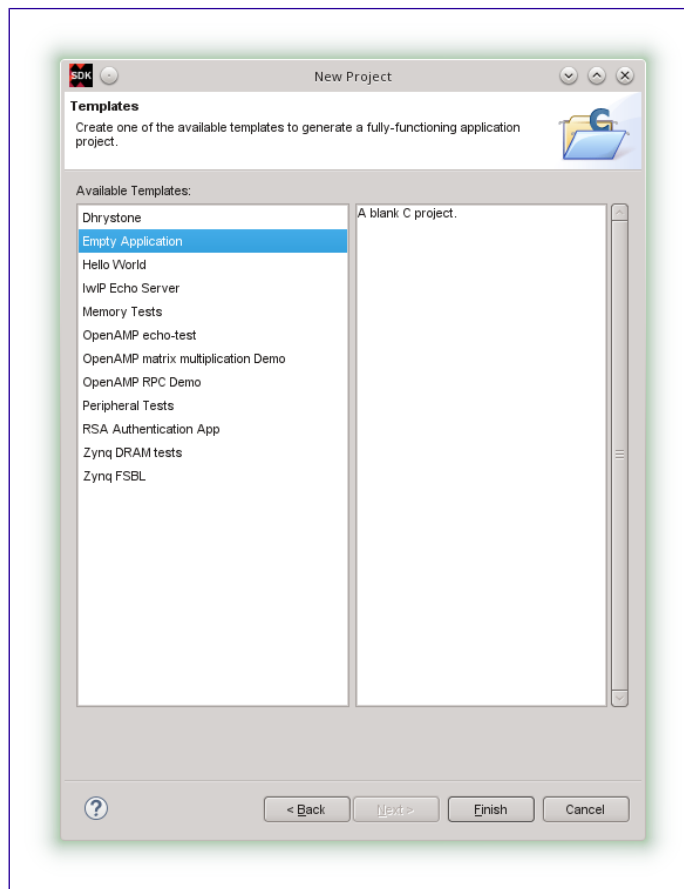


Fig 8.6: Empty Application Template

7. Now import all files from the “src” folder into the “src” directory of the Eclipse project.

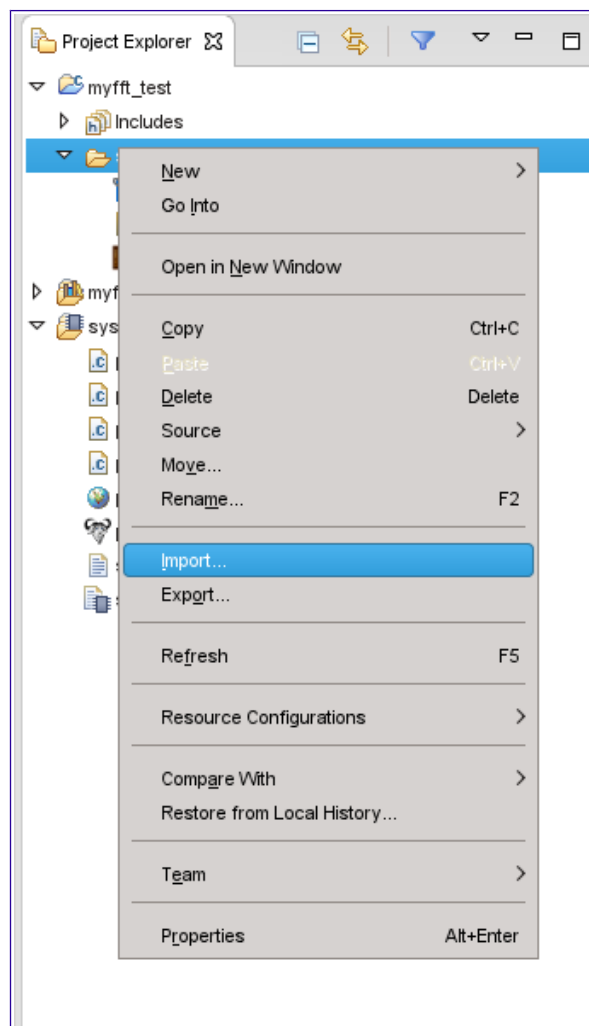


Fig 8.7: Import into the src directory

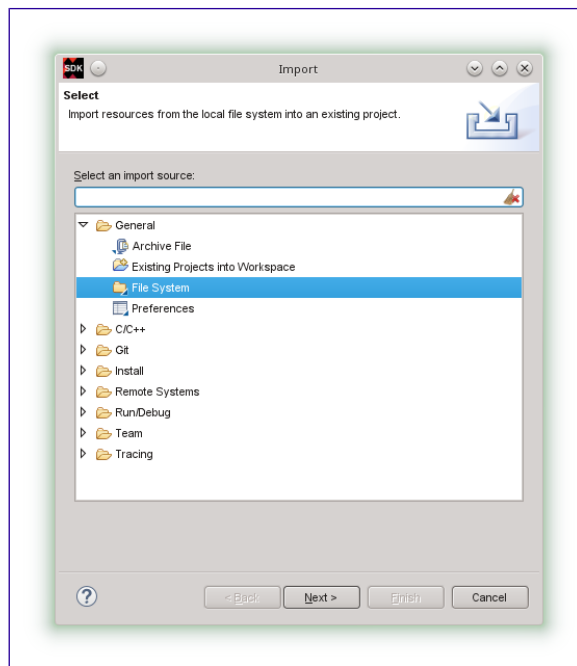


Fig 8.8: Import from File System

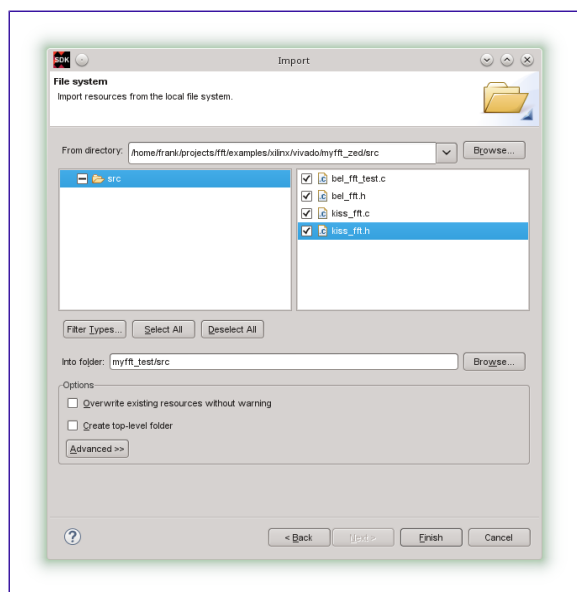


Fig 8.9: Import all Files

8. Program the FPGA

9. Start the program

8.2 | PCI-Express Example Design

This design shows how to use the FFT core together with a PCI-Express core and how to control it from a Linux application on a host PC. It has been designed and tested on a Xilinx AC701 board [AC701], but it should be easy to port it to other boards (e.g. KC705 [KC705], KCU105 [KCU105]).

The example design includes a Linux driver and two small test programs. The hardware design is completely created by a Tcl script. Building the design is done in five steps:

1. Build the hardware design
2. Program the configuration flash design
3. Put the board into your PC and boot your Linux
4. Build and run the driver
5. Build and run the test program

8.2.1 | Building the Hardware Design

The hardware design is created by a Vivado Tcl script. However, before you can start the script, you have to generate the FFT core.

Here is the detailed description:

1. The best is to copy the whole directory “myfft_pcie_ac701” to a place of your choice (e.g. “C:\Projects\myfft_pcie_ac701”).
2. Start the FFT wizard, open the configuration file “fft4096_32.xml” (*File*→*Open*), adjust the project directory so that the “myfft” directory is inside the “myfft_pcie_ac701” directory and then click *Generate*.
3. Now start Vivado and change to the “myfft_pcie_ac701” directory on the Tcl console (`cd C:/Projects/myfft_pcie_ac701`).
4. Start the Tcl Script (`source create_design.tcl`). This will generate the design, perform synthesis and implementation

and generate the bitstream and MCS file for the QSPI flash on the AC701 board. Figure 8.10 shows the generated hardware design.

5. Now connect the AC701 board and program the configuration flash device. This is done by sourcing the script “prog_spi.tcl”.

8.2.2 | Verifying the PCI-Express Design

Put the AC701 board into the PCI-Express slot of your Linux PC and boot your Linux. If you now issue the command “lspci” (as root), you should see the PCI-Express endpoint of the AC701 board:

```
...
00:18.4 Host bridge: Advanced Micro Devices, Inc. [AMD] Family
15h (Models 30h-3fh) Processor Function 4
00:18.5 Host bridge: Advanced Micro Devices, Inc. [AMD] Family
15h (Models 30h-3fh) Processor Function 5
01:00.0 Serial controller: Xilinx Corporation Device 7011
02:00.0 USB controller: ASMedia Technology Inc. Device 1343
...
```

8.2.3 | Building the Linux Driver and the Application

Program

The directory “belfft_pci” contains the Linux driver and two test programs for the PCI-Express example (short version and a more extensive version).

To get this to work you have to do the following things (as root):

1. Move to the driver directory


```
> cd belfft_pci
```
2. Do a cleanup


```
> make clean
```
3. Build the driver

- ```
> make
```
4. Install the driver
 

```
> insmod ./belfft_pci.ko
```
  5. Compile the test program
 

```
> gcc test_long.c
```
  6. Run the test program
 

```
> ./a.out
```

The output of the test program reports the calculated FFT results. If the output is

```
..: DEADBEAF - DEADBEAF
```

then the FFT could not DMA to its target buffer.

To view the debug output of the driver it is useful to have the command

```
> dmesg -w
```

running in a second terminal window.

The Linux driver creates a character device `/dev/belfft`. A user application opens this device, writes the input data to it, starts the FFT via an `ioctl` command and then reads back the calculated data.

This is only a basic example to show the basic principle of the interaction between the FFT core, the PCI-Express core, the Linux driver and the user application. Additional effort is required to create a production-ready solution.

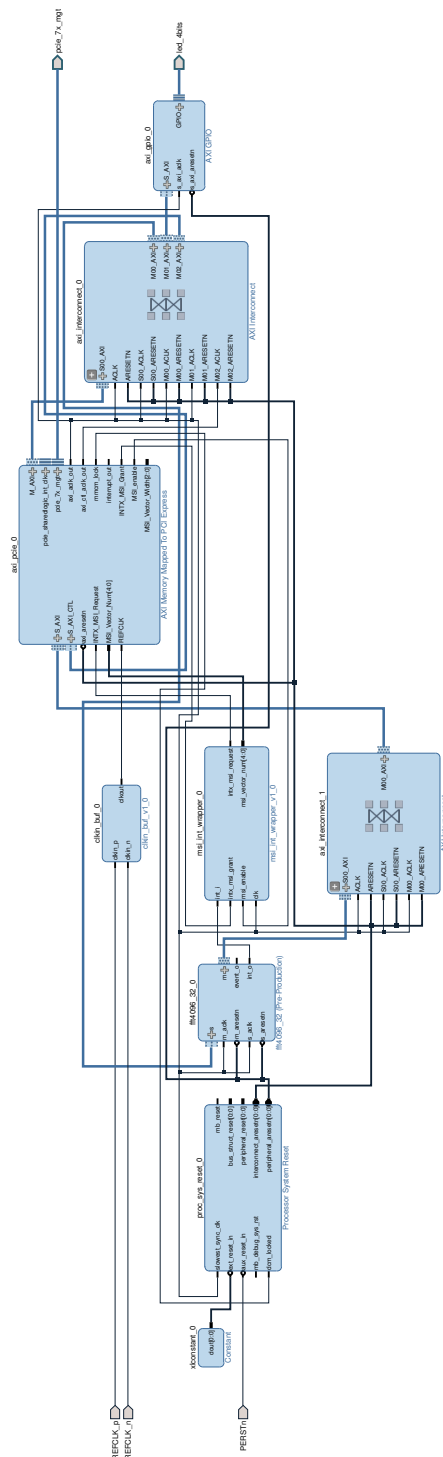


Fig 8.10: Block-design of PCI-Express system

## 9 | Integration in QSYS-BUILDER

The configuration wizard creates hardware Tcl file for the integration in QSYS. The file is located in the project directory and file name has the form `<module name>_hw.tcl`.

In order for QSYS to find the file it is necessary to add the project directory to the *IP Search Path* which can be configured in the *Options* menu of QSYS and the SOPC builder.

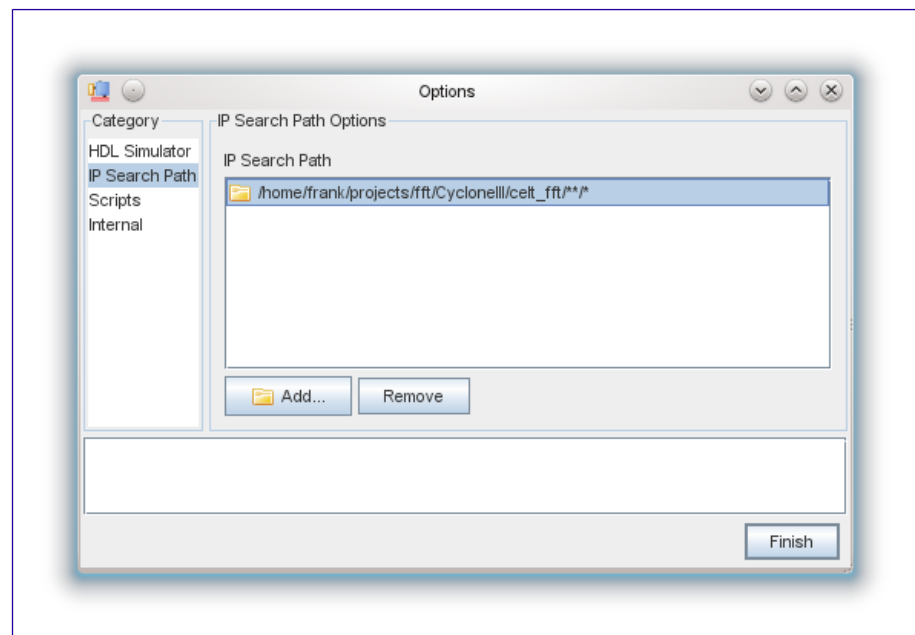


Fig 9.1: Setting the IP Search Path

In this case we have give the module the name `celt_fft`

After the project path has been added to the IP search path, the newly configured module can be found under the DSP category of the Component Library.

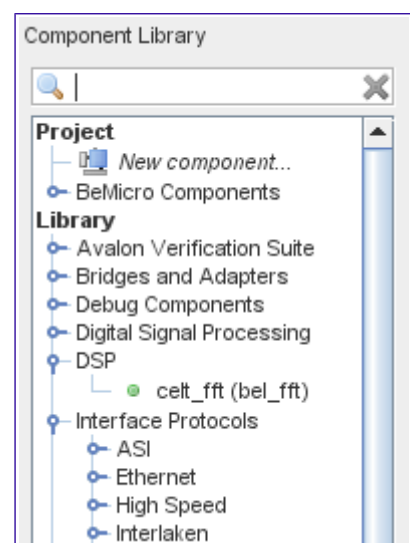
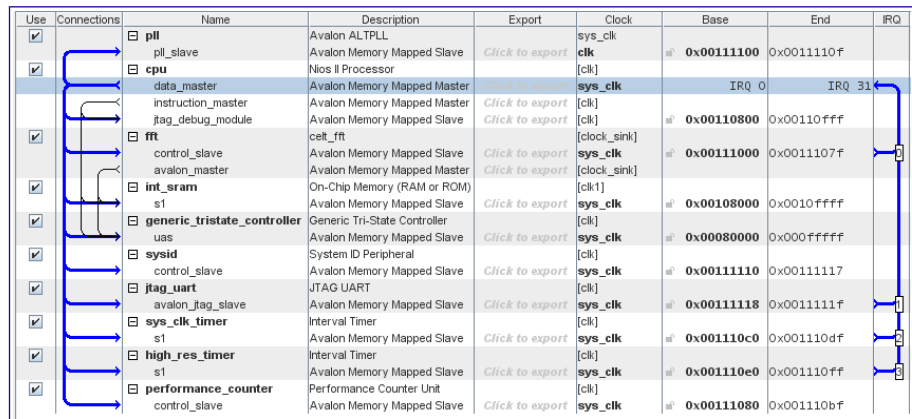


Fig 9.2: Component Library

In the picture below you can find a small example system which has been set up to run the low latency audio codec Celt [CELT] on the Be-Micro development board from Arrow [BEMICRO]. The FFT co-processor has been configured for two different configurations (256 and 64 point).



| Use                                 | Connections | Name                               | Description                  | Export                          | Clock        | Base       | End        | IRQ |
|-------------------------------------|-------------|------------------------------------|------------------------------|---------------------------------|--------------|------------|------------|-----|
| <input checked="" type="checkbox"/> |             | <b>pll</b>                         | Avalon ALTPLL                |                                 | sys_clk      |            |            |     |
|                                     |             | pll_slave                          | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | clk          | 0x00111100 | 0x0011110f |     |
| <input checked="" type="checkbox"/> |             | <b>cpu</b>                         | Nios II Processor            |                                 |              |            |            |     |
|                                     |             | data_master                        | Avalon Memory Mapped Master  | <a href="#">Click to export</a> | sys_clk      | IRQ 0      | IRQ 31     |     |
|                                     |             | instruction_master                 | Avalon Memory Mapped Master  | <a href="#">Click to export</a> | clk          |            |            |     |
|                                     |             | jtag_debug_module                  | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | clk          | 0x00110800 | 0x00110fff |     |
| <input checked="" type="checkbox"/> |             | <b>fft</b>                         | celt_fft                     |                                 | [clock_sink] |            |            |     |
|                                     |             | control_slave                      | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | sys_clk      | 0x00111000 | 0x0011107f |     |
|                                     |             | avalon_master                      | Avalon Memory Mapped Master  | <a href="#">Click to export</a> | [clock_sink] |            |            |     |
| <input checked="" type="checkbox"/> |             | <b>int_sram</b>                    | On-Chip Memory (RAM or ROM)  |                                 | [clk1]       |            |            |     |
|                                     |             | s1                                 | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | sys_clk      | 0x00108000 | 0x0010ffff |     |
| <input checked="" type="checkbox"/> |             | <b>generic_tristate_controller</b> | Generic Tri-State Controller |                                 | clk          |            |            |     |
|                                     |             | uas                                | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | sys_clk      | 0x00080000 | 0x000fffff |     |
| <input checked="" type="checkbox"/> |             | <b>sysid</b>                       | System ID Peripheral         |                                 | clk          |            |            |     |
|                                     |             | control_slave                      | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | sys_clk      | 0x00111110 | 0x00111117 |     |
| <input checked="" type="checkbox"/> |             | <b>jtag_uart</b>                   | JTAG UART                    |                                 | clk          |            |            |     |
|                                     |             | avalon_jtag_slave                  | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | sys_clk      | 0x00111118 | 0x0011111f |     |
| <input checked="" type="checkbox"/> |             | <b>sys_clk_timer</b>               | Interval Timer               |                                 | clk          |            |            |     |
|                                     |             | s1                                 | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | sys_clk      | 0x001110c0 | 0x001110df |     |
| <input checked="" type="checkbox"/> |             | <b>high_res_timer</b>              | Interval Timer               |                                 | clk          |            |            |     |
|                                     |             | s1                                 | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | sys_clk      | 0x001110e0 | 0x001110ff |     |
| <input checked="" type="checkbox"/> |             | <b>performance_counter</b>         | Performance Counter Unit     |                                 | clk          |            |            |     |
|                                     |             | control_slave                      | Avalon Memory Mapped Slave   | <a href="#">Click to export</a> | sys_clk      | 0x00111080 | 0x001110bf |     |

Fig 9.3: QSYS system

The configuration interface is connected to the data master port of the NiosII processor so that it can be configured. The memory interface is connected to the internal SRAM (onchip\_sram) and the external static memory (generic\_tristate\_controller). The interrupt signal is connected to the processor.

## 10 | Manual ROM Generation

Before you generate the twiddle factor ROMs manually, generate all other files.

Due to bugs and changing interfaces, the automatic twiddle factor ROM generation does not work with all software versions. So here is still a description how the twiddle factor ROMs can be generated manually.

### 10.1 | Altera Quartus

In Quartus you have to use the Mega Wizard to generate the ROMs. The name of the generated ROM has to be

`<module name>_twiddle_rom<i>`

where `<i>` is the configuration index starting at 0.

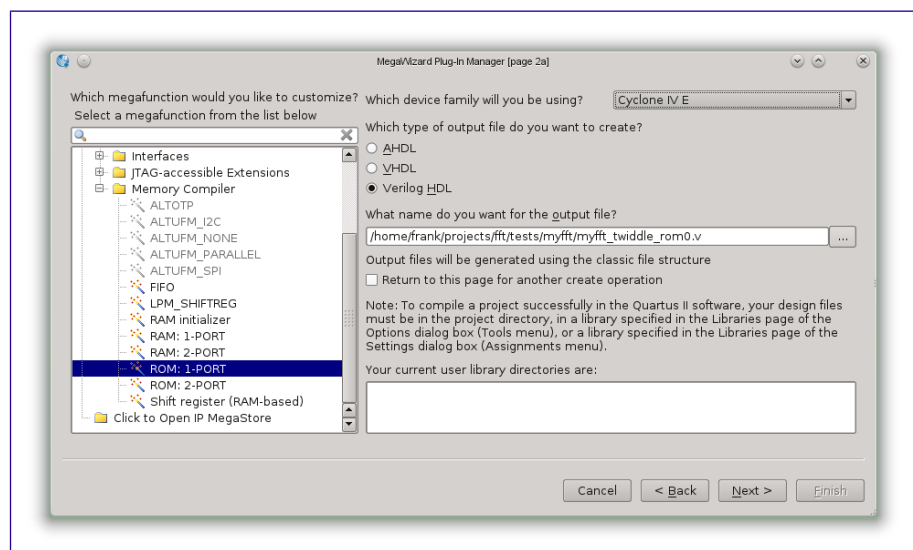


Fig 10.1: Selection of Mega Function

Select the ROM 1-Port Mega Function and enter the target name.

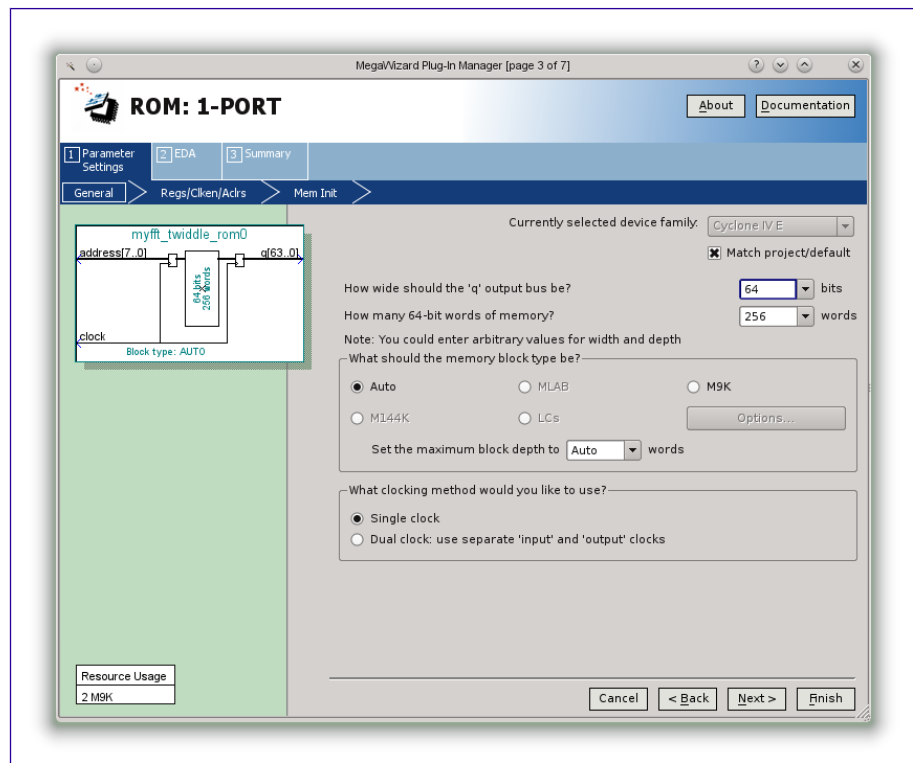


Fig 10.2: Selection size and interface width

In the next dialogue enter the size of the ROM. This is the size of the FFT configuration. The width of the output 'q' has to be two times the size of the selected word size (64 bit for a 32 bit FFT).

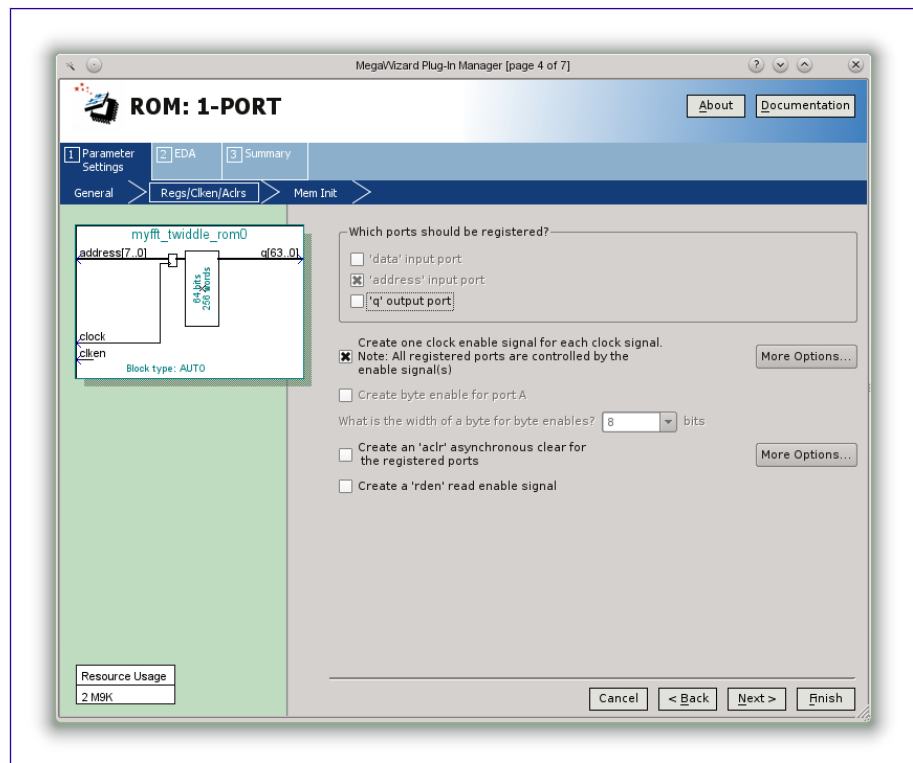


Fig 10.3: Enabling clock enable

In the next step the clock enable signal has to be enabled and the output register for the ROM has to be disabled.

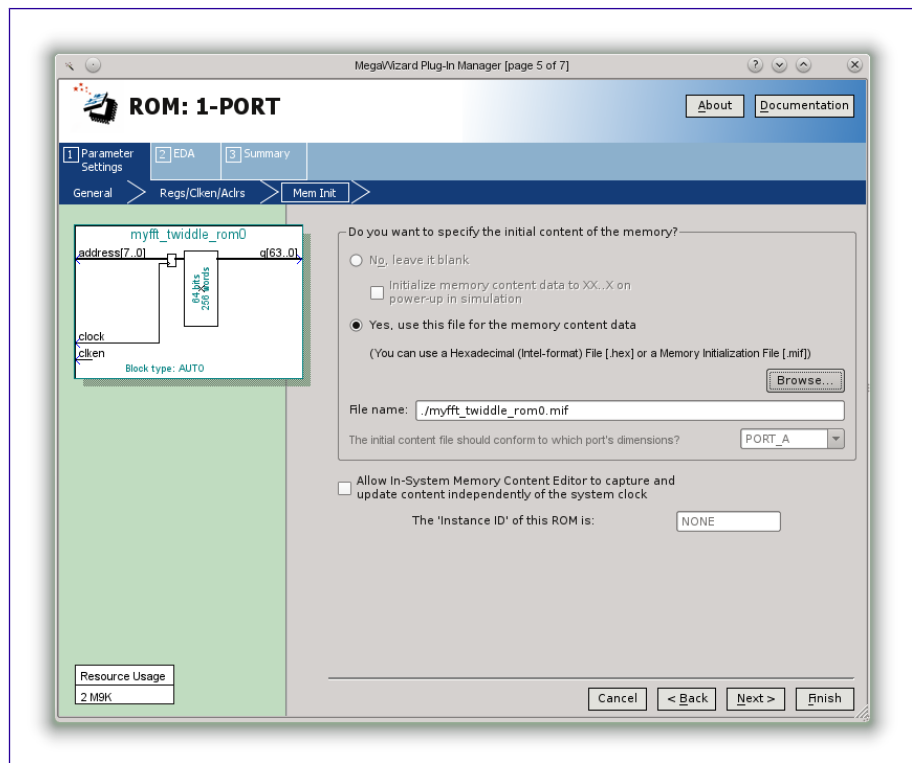


Fig 10.4: Selecting the ROM contents file

Now the ROM contents has to be selected. This is the MIF file previously generated.

You can step through the remaining dialogues and finally select finish. This will generate the ROM for one configuration. Repeat this for all other configurations.

## 11 | Software interface

### 11.1 | Direct programming

The following is an example of how to program the co-processor. This is completely encapsulated inside the Kiss API, so it is recommended to use the Kiss API instead of programming the registers directly.

```

/*
 * e.g. an fft of length 128 has 4 factors
 * as far as kissfft is concerned
 * 4*4*4*2
 */
#define MAXFACTORS 32

/* Control register */
struct ControlReg {
 /* Starts the FFT */
 int Start: 1;
 int reserved1: 7;
 /* Enables the interrupt */
 int Inten: 1;
 int reserved2: 7;
 /* Normal or inverse FFT */
 int Inv: 1;
 int reserved3: 15;
};

/* Status register */
struct StatusReg {
 /* Indicates that the FFT is running. */
 int Running: 1;
 /* Overflow flag */
 int Ov: 1;
 /* Interrupt flag. This flag is cleared after reading. */
 int Int: 1;
 /* Error flag. This flag indicates that an error occurred
while

```

```

 * the FFT was accessing the bus. This flag is cleared af-
 ter
 * reading. */
 int Err: 1;
 int reserved1: 28;
};

/* FFT size register */
struct NReg {
 /* Size of the FFT */
 int N: 16;
 int reserved1: 16;
};

/* FFT size register */
struct FactorsReg {
 /* Stage's FFT length/p */
 int M: 16;
 /* The radix */
 int P: 16;
};

/* Register structure, must be mapped to base address */
struct bel_fft {
 /* Control register */
 struct ControlReg Control;
 /* Status register */
 struct StatusReg Status;
 /* FFT size register */
 struct NReg N;
 /* Source address register */
 void * Finadr;
 /* Destination address register */
 void * Foutadr;
 /* FFT factor registers */
 struct FactorsReg Factors[MAXFACTORS];
};

```

```

#define FFT_LEN 256

int main() {

 /* FFT_BASE is the base address of the FFT
 * co-processor.
 * Set bit 31 to bypass the cache on the NIOSII.
 */

 volatile struct bel_fft * belFftPtr = (struct bel_fft *)
(FFT_BASE + 0x80000000);

 int fin[FFT_LEN * 2] = {
 0x00000000, 0x00000000, 0x00002BD1, 0x00000000,
 0x000040E8, 0x00000000, 0x000035CE, 0x00000000,
 ...

 int fout[FFT_LEN * 2];

 belFftPtr->Finadr = fin;

 belFftPtr->Foutadr = fout;

 belFftPtr->Factors[0].M = 64;
 belFftPtr->Factors[0].P = 4;
 belFftPtr->Factors[1].M = 16;
 belFftPtr->Factors[1].P = 4;
 belFftPtr->Factors[2].M = 4;
 belFftPtr->Factors[2].P = 4;
 belFftPtr->Factors[3].M = 1;
 belFftPtr->Factors[3].P = 4;

 belFftPtr->Control.Start = 1;
 while (! cfg->belFftPtr->Status.Int) {}

```

## 11.2 | Kiss API

Here is the same FFT by using the Kiss API.

```
#define FFT_LEN 256

int main() {

 kiss_fft_cpx fin[FFT_LEN] = {
 {0x00000000, 0x00000000}, {0x00002BD1, 0x00000000},
 {0x000040E8, 0x00000000}, {0x000035CE, 0x00000000},
 ...

 kiss_fft_cpx fout[FFT_LEN];

 kiss_fft_cfg cfg;

 cfg = kiss_fft_alloc (FFT_LEN, 0, NULL, NULL);

 kiss_fft (cfg, fin, fout);
```

## 11.3 | Aligning Input and Output Data

If the FFT co-processor uses a 64-bit interface, input and output data has to be carefully aligned to a 64 bit boundary.

```
kiss_fft_cpx __attribute__((aligned(8))) fin[FFT_LEN] = {
 {0x00000000, 0x00000000}, {0x00002BD1, 0x00000000},
 {0x000040E8, 0x00000000}, {0x000035CE, 0x00000000},
 ...

 kiss_fft_cpx __attribute__((aligned(8))) fout[FFT_LEN];
```

## 11.4 | Settings for Cortex-A9 ACP Port

When the master interface of the FFT co-processor is connected to the ACP port of a Cortex-A9 then the settings of the user and cache signals of the AXI bus interface become important. For the settings of these signals the co-processor provides a special user register at offset 0x0FF8. When using the Kiss API, the settings of the values have to be changed in the function **kiss\_fft\_stride**.

The default value is 0x0F010F01.

## 12 | Simulation

### 12.1 | Simulation with ISim and XSim

Scripts for simulation with Xilinx ISim and XSim are only generated when Xilinx is the target technology. In contrast to the ModelSim setup script (see below) the generated Tcl [TCL] scripts are not called from within the simulator but from the command line. They copy all needed auxiliary files into the simulation directory, compile all files, and build a snapshot that is started for simulation.

### 12.2 | Simulation with ModelSim

The configuration wizard creates a simulation environment for ModelSim [MODELALT]. A free version of ModelSim, the ModelSim Starter Edition is available from the Altera web-site.

#### 12.2.1 | Setup

Under each project directory there is a simulation directory for each configuration. In each simulation directory a testbench, an input data file, a generic RAM model and a setup script for ModelSim can be found.

1. Start ModelSim
2. Check the current directory (by typing `pwd` on the ModelSim command line). If the current directory is not the simulation directory, change to it (e.g. `cd /home/frank/projects/fft/CycloneIII/celt_fft/simulation_256`).
3. Source the `setup.tcl` file (`source setup.tcl`) in the ModelSim command line). `setup.tcl` defines two shortcuts for compiling all files ("c") and for starting the simulation ("s").
4. Type in "c"
5. Now type in "s"

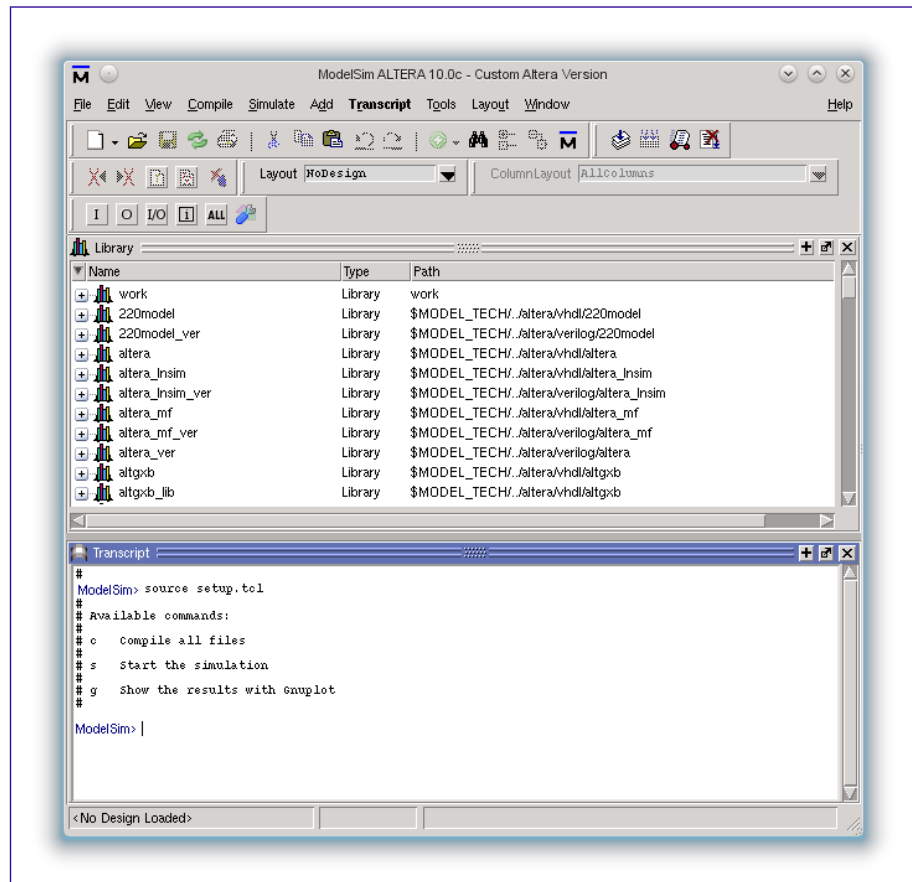


Fig 12.1: ModelSim after sourcing setup.tcl

### 12.2.2 | Running the Simulation

The testbench contains two memories, one providing the input data and one collecting the output data and writing it to a file. The testbench is not self checking, which means you cannot see from the simulation whether the FFT worked correctly. Instead, the contents of the RAM has to be analyzed.

After the simulation view has started, ModelSim also opens a wave window with all testbench signals. You can add your own signals of interest to the wave window before starting the simulation. The setup script also defines mnemonics for the states of the state machines.

The testbench will always terminate, either because the `bel_fft` module triggers an interrupt or because a timeout occurs.

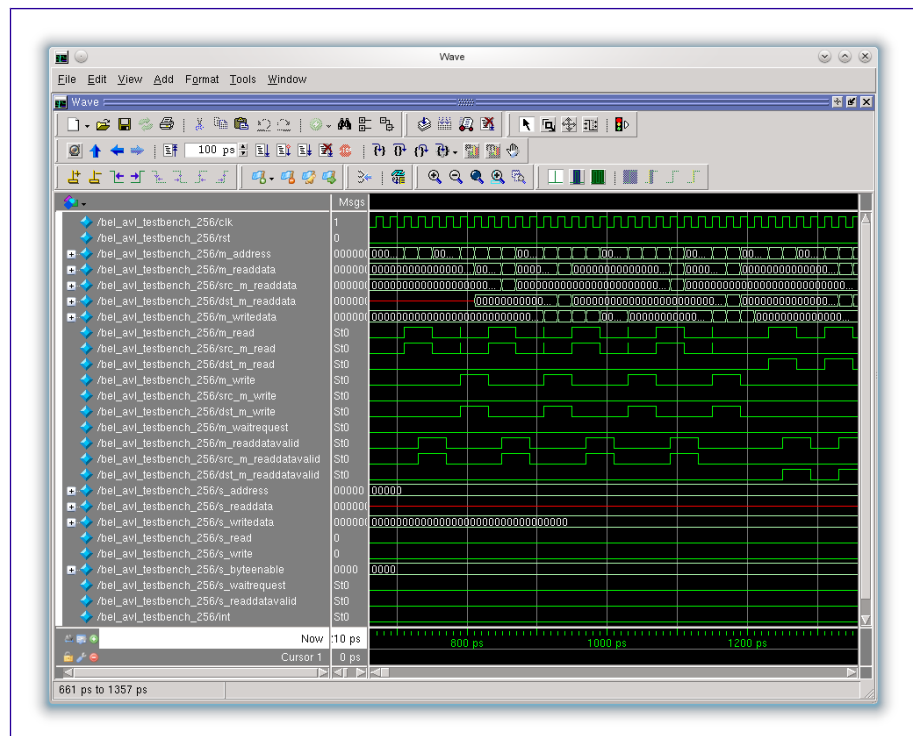


Fig 12.2: Simulation results

### 12.2.3 | Viewing the Results

The `setup.tcl` script also provides a function which splits the resulting output data into a real and imaginary part and displays the data with Gnuplot [GNUPLLOT]. If Gnuplot is installed it can be started by simply typing “g”. The real part is shown in red, the imaginary part in green.

There is also a standalone Tcl script `view-results.tcl` which does the same outside of the simulator.

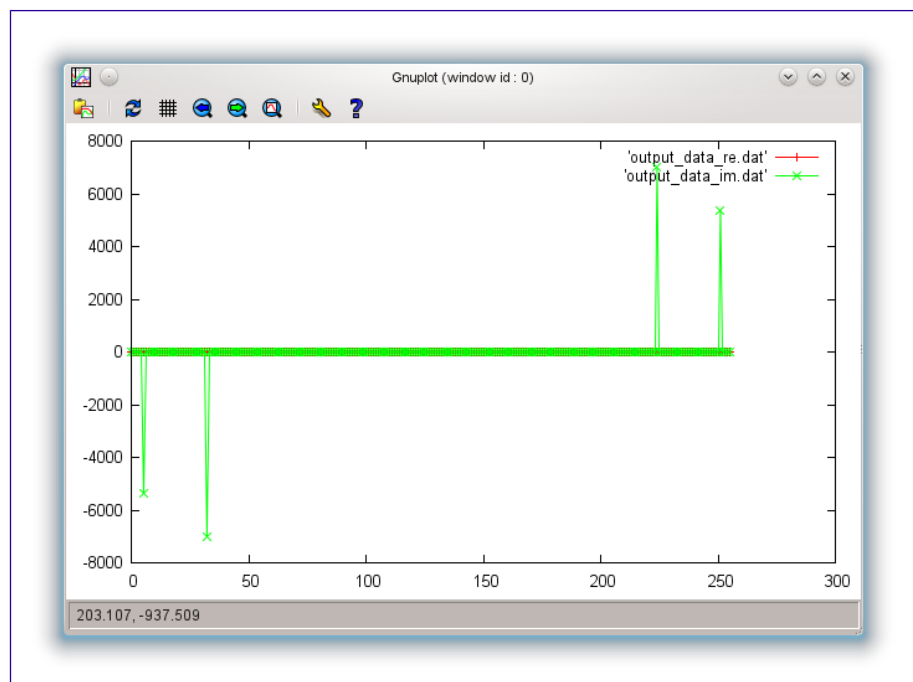


Fig 12.3: Viewing the simulation results with Gnuplot

#### 12.2.4 | Running Simulation from an Instance inside a Vivado Block Design

Running the simulation of the FFT processor from an instance inside a Vivado block design is easy. Simply select the instance, and then call the function *Edit in IP Packager* from the popup menu.

This will open a new project for the FFT processor. Inside the new project, you simply start the simulation (*Simulation*→*Run Simulation*→*Run Behavioral Simulation*).

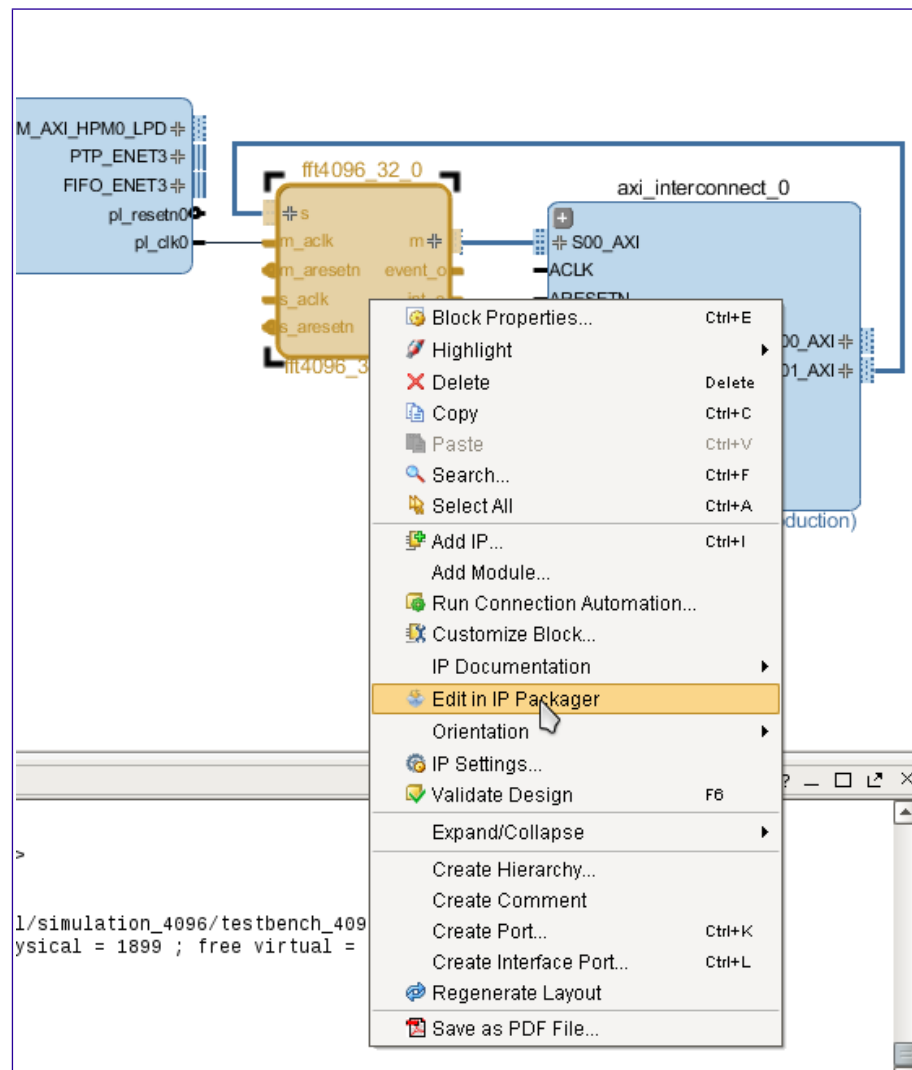


Fig 12.4: Edit in IP Packager

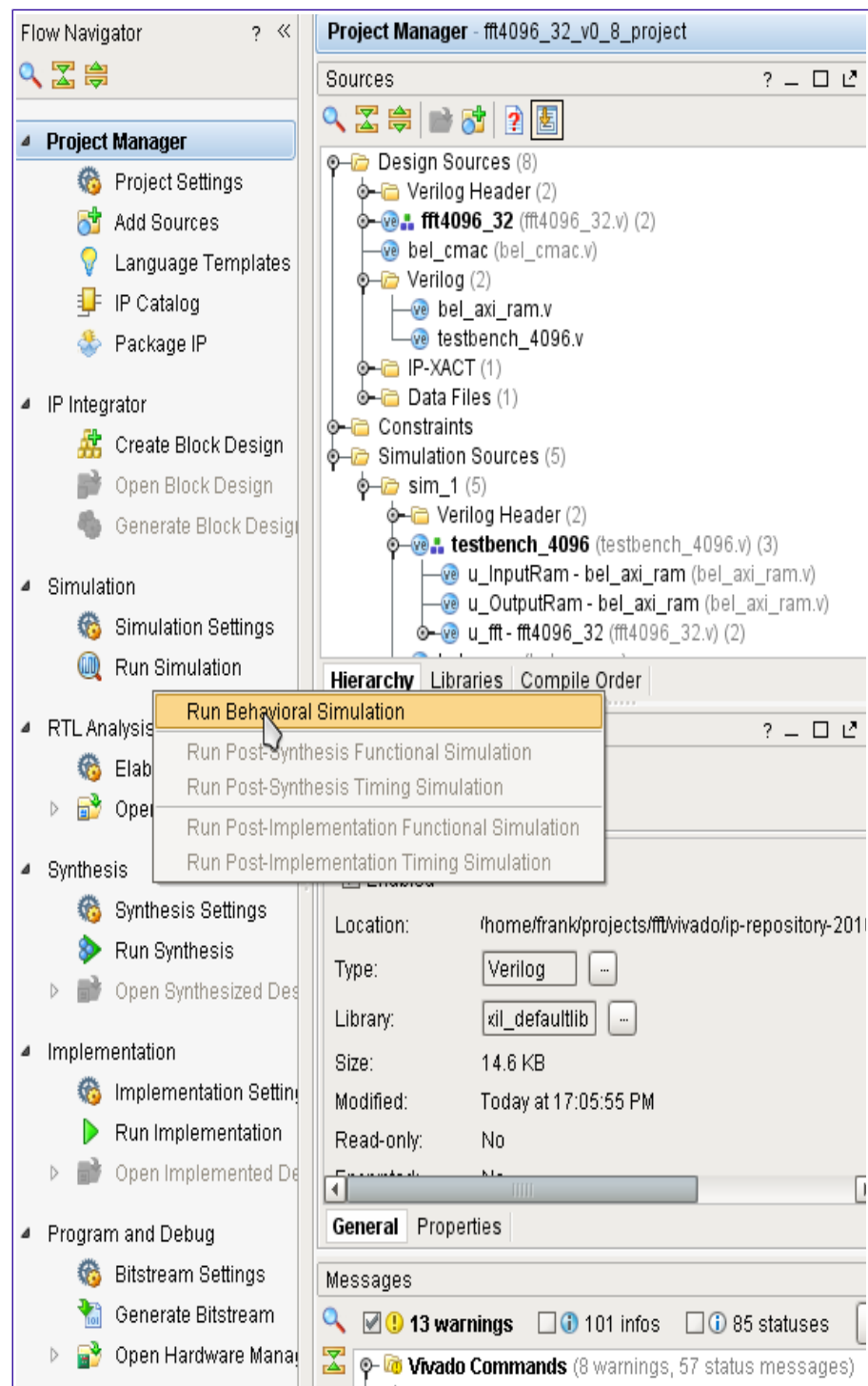


Fig 12.5: Run Behavioral Simulation

## 13 | Performance

### 13.1 | Altera CycloneIII Implementation

The performance measurements have been performed on an Altera CycloneIII EP3C16F256C8N (BeMicro). The FFT had a size of 256 bits with a data representation of 32 bit. Program and data was stored on the external SRAM of the BeMicro (512KB).

Performance was measured and reported on the NiosII by measuring the ticks of a counter that was clocked by the system clock (48MHz).

| FFT Hardware                                                             | Ticks     | Factor |
|--------------------------------------------------------------------------|-----------|--------|
| bel_fft, 48MHz                                                           | 74.539    | 1      |
| NiosII fast (default configuration, 4K instruction cache, 4K data cache) | 1.694.588 | 22.7   |

### 13.2 | Xilinx Zynq Implementation

On a ZedBoard (XC7Z020-1CLG484C) the FFT core can run at 100MHz (FFT size of 256 and 32 bit word size).

| Resource | Utilization |
|----------|-------------|
| LUT      | 2297        |
| Register | 2226        |
| DSP      | 17          |
| BRAM     | 1           |

## 14 | Setting Path Variable in Windows 7

Here is still the description of how to set the Path environment variable under Windows 7.

In the Control Panel, open the *System* section. There, select *Advanced system settings*.

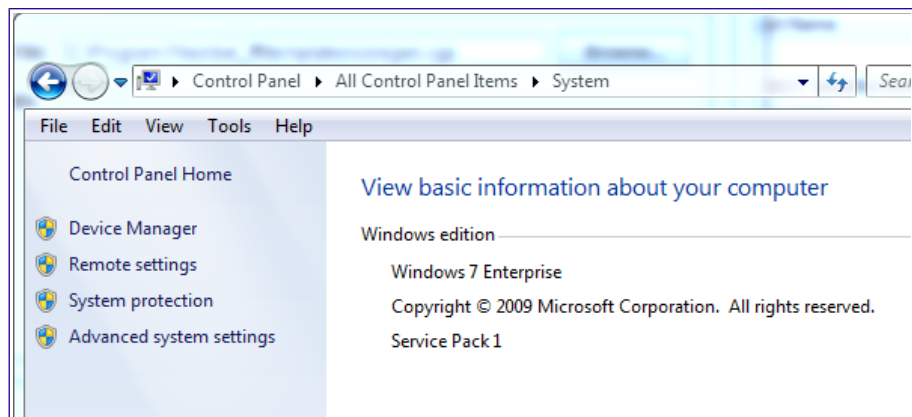


Fig 14.1: System settings

In the Advanced system settings, select *Environment Variables...*

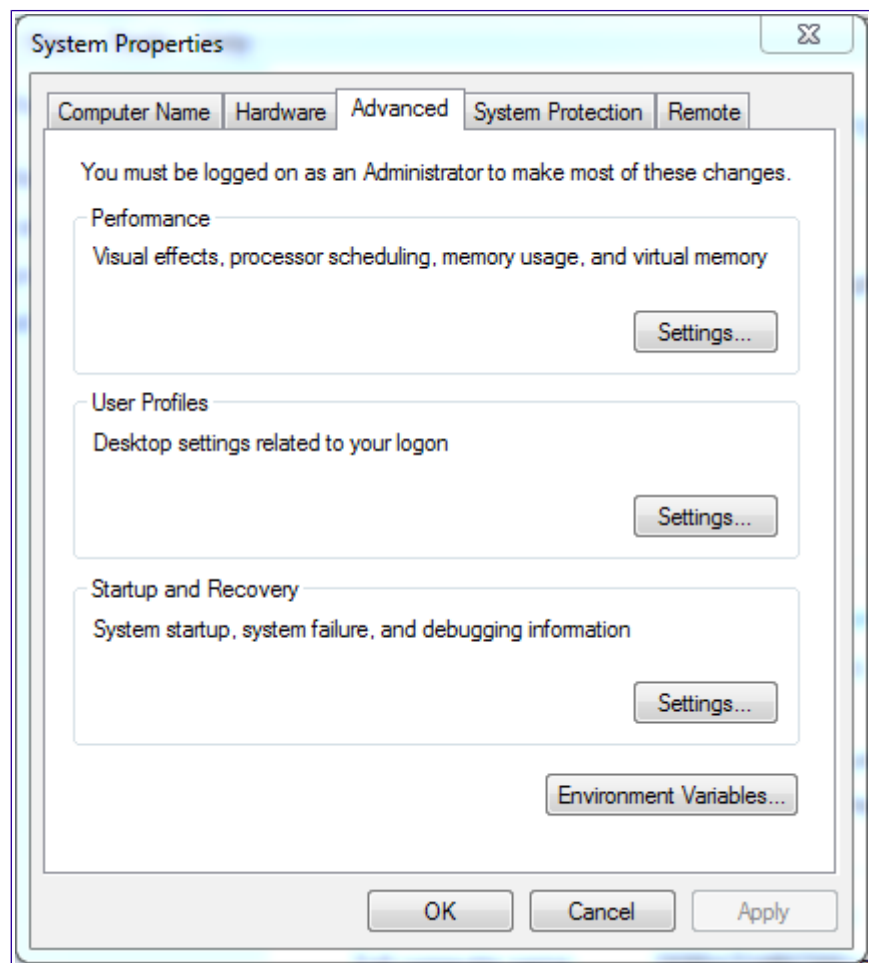


Fig 14.2: Advanced system settings

The following dialogue lists the user variables and the system variables. You should already have a user variable Path, which has to be expanded.

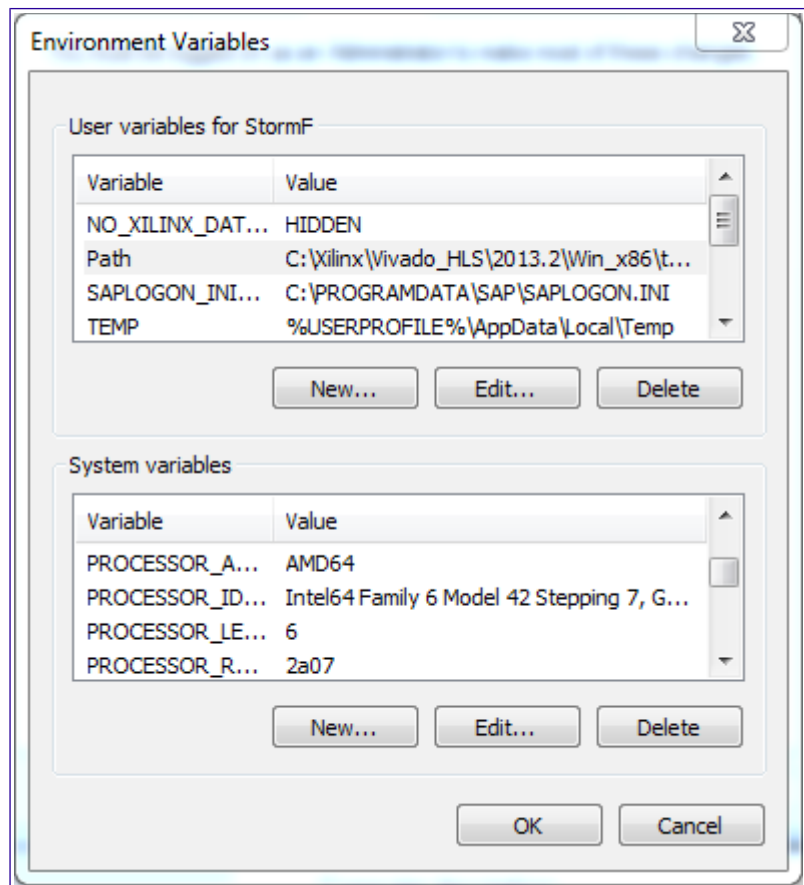


Fig 14.3: Environment Variables

Select the Path variable and click *Edit...* Then prepend the location of the required programs (e.g. C:\Xilinx\ISE\_DS\ISE\bin\nt64 for core-gen under Windows 7, 64-bit) and separate it from the other directories with a semicolon.

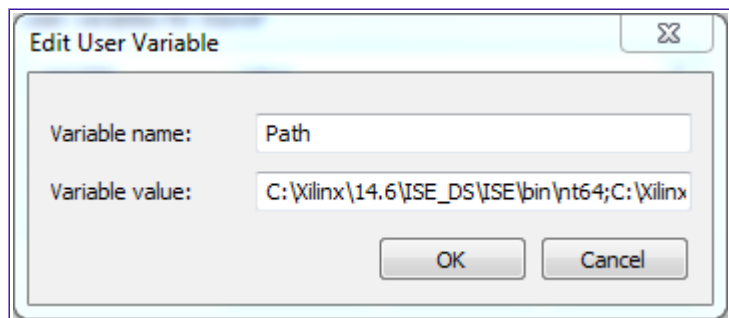


Fig 14.4: Edit Path variable

## **15 | GNU LESSER GENERAL PUBLIC LICENSE**

### **15.1 | Version 2.1, February 1999**

Copyright (C) 1991, 1999 Free Software Foundation, Inc.

51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

[This is the first released version of the Lesser GPL. It also counts as the successor of the GNU Library Public License, version 2, hence the version number 2.1.]

### **15.2 | Preamble**

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licenses are intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users.

This license, the Lesser General Public License, applies to some specially designated software packages--typically libraries--of the Free Software Foundation and other authors who decide to use it. You can use it too, but we suggest you first think carefully about whether this license or the ordinary General Public License is the better strategy to use in any particular case, based on the explanations below.

When we speak of free software, we are referring to freedom of use, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish); that you receive source code or

can get it if you want it; that you can change the software and use pieces of it in new free programs; and that you are informed that you can do these things.

To protect your rights, we need to make restrictions that forbid distributors to deny you these rights or to ask you to surrender these rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link other code with the library, you must provide complete object files to the recipients, so that they can relink them with the library after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

We protect your rights with a two-step method: (1) we copyright the library, and (2) we offer you this license, which gives you legal permission to copy, distribute and/or modify the library.

To protect each distributor, we want to make it very clear that there is no warranty for the free library. Also, if the library is modified by someone else and passed on, the recipients should know that what they have is not the original version, so that the original author's reputation will not be affected by problems that might be introduced by others. Finally, software patents pose a constant threat to the existence of any free program. We wish to make sure that a company cannot effectively restrict the users of a free program by obtaining a restrictive license from a patent holder. Therefore, we insist that any patent license obtained for a version of the library must be consistent with the full freedom of use specified in this license.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License. This license, the GNU Lesser General Public License, applies to certain designated libraries, and is quite different from the ordinary General Public License. We use this license for certain libraries in order to permit linking those libraries into non-free programs.

When a program is linked with a library, whether statically or using a shared library, the combination of the two is legally speaking a combined work, a derivative of the original library. The ordinary General Public License therefore permits such linking only if the entire combination fits its criteria of freedom. The Lesser General Public License permits more lax criteria for linking other code with the library.

We call this license the "Lesser" General Public License because it does Less to protect the user's freedom than the ordinary General Public License. It also provides other free software developers Less of an advantage over competing non-free programs. These disadvantages are the reason we use the ordinary General Public License for many libraries. However, the Lesser license provides advantages in certain special circumstances.

For example, on rare occasions, there may be a special need to encourage the widest possible use of a certain library, so that it becomes a de-facto standard. To achieve this, non-free programs must be allowed to use the library. A more frequent case is that a free library does the same job as widely used non-free libraries. In this case, there is little to gain by limiting the free library to free software only, so we use the Lesser General Public License.

In other cases, permission to use a particular library in non-free programs enables a greater number of people to use a large body of free software. For example, permission to use the GNU C Library in non-free programs enables many more people to use the whole GNU operating system, as well as its variant, the GNU/Linux operating system.

Although the Lesser General Public License is Less protective of the users' freedom, it does ensure that the user of a program that is linked with the Library has the freedom and the wherewithal to run that program using a modified version of the Library.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a "work based on the library" and a "work that uses the library". The former

contains code derived from the library, whereas the latter must be combined with the library in order to run.

### **15.3 | TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION**

0. This License Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Lesser General Public License (also called "this License"). Each licensee is addressed as "you".

A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

"Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

**1.** You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

**2.** You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

- **a)** The modified work must itself be a software library.
- **b)** You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.
- **c)** You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.
- **d)** If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

(For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can

be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it. Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

**3.** You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

**4.** You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

**5.** A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

**6.** As an exception to the Sections above, you may also combine or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under

terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

- **a)** Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)
- **b)** Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (1) uses at run time a copy of the library already present on the user's computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.
- **c)** Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.

- **d)** If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.
- **e)** Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

**7.** You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

- **a)** Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.
- **b)** Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

**8.** You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Li-

brary is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

**9.** You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

**10.** Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties with this License.

**11.** If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to

apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

**12.** If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

**13.** The Free Software Foundation may publish revised and/or new versions of the Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

**14.** If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted

by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

## **NO WARRANTY**

**15.** BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

**16.** IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

## **END OF TERMS AND CONDITIONS**



## 16 | License of KISS FFT

The KISS FFT is provided under the following license:

Copyright (c) 2003-2010 Mark Borgerding

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the author nor the names of any contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT

OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE  
POSSIBILITY OF SUCH DAMAGE.

## Literature

KISS: Marc Borgerding, Keep it simple, Stupid FFT, , <http://sourceforge.net/projects/kissfft/>

AXI: ARM, AMBA AXI and ACE Protocol Specification, 2011

AVALON: Altera, Avalon Interface Specifications, 2011, [http://www.altera.com/literature/manual/mnl\\_avalon\\_spec.pdf](http://www.altera.com/literature/manual/mnl_avalon_spec.pdf)

WISHBONE: OpenCores, Wishbone B4 - Wishbone System-on-chip (SoC) Interconnection Architecture for portble IP Cores, 2010, [http://opencores.org/cdn/downloads/wbspec\\_b4.pdf](http://opencores.org/cdn/downloads/wbspec_b4.pdf)

AMBA: ARM, AMBA™ Specification (Rev. 2.0), 13th May 1999

XPS: Xilinx, Xilinx Platform Studio, , <http://www.xilinx.com/tools/xps.htm>

QSYS: Altera, QuartusII Handbook Version 11.0 Volume1: Design and Synthesis Section II: System Design with QSYS, 2011, [http://www.altera.com/literature/hb/qts/qsys\\_section.pdf](http://www.altera.com/literature/hb/qts/qsys_section.pdf)

NETBEANS: Oracle, NetBeans IDE, 7.0, <http://netbeans.org>

MIF: Altera, Memory Initialization File (.mif) Definition, 11.0, [http://quartushelp.altera.com/11.0/mergedProjects/reference/glossary/def\\_mif.htm](http://quartushelp.altera.com/11.0/mergedProjects/reference/glossary/def_mif.htm)

AR53695: Xilinx, 14.4 CORE Generator - IP core cannot be created when using a WebPACK tools install, 8th Jan 2013, <http://www.xilinx.com/support/answers/53695.htm>

AC701: Xilinx, , , <https://www.xilinx.com/products/boards-and-kits/ek-a7-ac701-g.html>

KC705: Xilinx, , , <https://www.xilinx.com/products/boards-and-kits/ek-k7-kc705-g.html>

KCU105: , , , <https://www.xilinx.com/products/boards-and-kits/kcu105.html>

CELT: the xiph open source community, The CELT ultra-low delay audio codec, 0.11.1, <http://www.celt-codec.org/>

BEMICRO: Arrow, BeMicro Evaluation Board - A compact, Low-cost FPGA-based Microcontroller Evaluation Platform, 2011,

<http://www.arroweurope.com/markets-solutions/solutions/bemicro/bemicro.html>

TCL: , Tcl/Tk, , <http://tcl.sourceforge.net/>

MODELALT: Altera, ModelSim-Altera Software, 6.6d, <http://www.altera.com/products/software/quartus-ii/modelsim/qts-modelsim-index.html>

GNUPLOT: Thomas Williams, Colin Kelley, Gnuplot, 4.4.4, [www.gnuplot.info](http://www.gnuplot.info)